

FlagEmbedding



BGE: One-Stop Retrieval Toolkit For Search and RAG

FlagEmbedding (BAAI)

01 overview

FlagEmbedding 简介



FlagEmbedding (BGE) 简介

Mermaid Diagram

1.		
2.		
3.		
4.		
5.		
6.		

1.

1.1 FlagEmbedding

FlagEmbedding EmbeddingRerankerBAI

1.2

- BAI
- BGE (BAI General Embedding)
-
-
- Encoder-only Decoder-only
-
-

1.3

FlagEmbedding  -  -  -  - 


2.

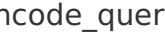
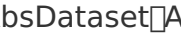
2.1

FlagEmbedding 

1.  **abc**/
2.  **inference**/
3.  **finetune**/
4.  **evaluation**/

2.2

ABC

- [AbsEmbedder](#) encode encode_queries encode_corpus 
- [AbsReranker](#)
-  AbsModeling AbsDataset AbsTrainer AbsRunner

 Encoder-only  Decoder-only

2.3

 [FlagAutoModel](#)  [FlagAutoReranker](#)  -  - 
HuggingFace Hub  - 

2.4

[AbsEmbedder](#)  -  CUDA NPU MUSA MPS CPU -  - 
OOM 

3.

```

/workspace/FlagEmbedding/
├─ abc/                                     # 数据集
│   └─ inference/                         # 推理
│       └─ AbsEmbedder.py                # 抽象基类
│       └─ AbsReranker.py                # 抽象基类
│       └─ __init__.py
│   └─ finetune/                          # 微调
│       └─ embedder/                     # 嵌入
│       └─ reranker/                     # 重排
│       └─ __init__.py
│   └─ evaluation/                        # 评估
│       └─ __init__.py
├─ inference/                             # 推理
│   └─ auto_embedder.py                  # 自动嵌入
│   └─ auto_reranker.py                  # 自动重排
│   └─ embedder/                         # 嵌入
│       └─ encoder_only/                 # Encoder-only
│           └─ base.py                   # 基类
│           └─ m3.py                     # BGE-M3
│       └─ decoder_only/                 # Decoder-only
│           └─ base.py
│           └─ icl.py                     # 指令
│           └─ pseudo_moe.py             # MoE
│       └─ model_mapping.py              # 模型映射
│       └─ __init__.py
│   └─ reranker/                         # 重排
│       └─ encoder_only/
│       └─ decoder_only/
│       └─ model_mapping.py
│       └─ __init__.py
│   └─ __init__.py
├─ finetune/                             # 微调
│   └─ embedder/                         # 嵌入
│   └─ reranker/                         # 重排
│   └─ __init__.py
├─ evaluation/                           # 评估
│   └─ mteb/                             # MTEB
│   └─ beir/                             # BEIR
│   └─ msmarco/                          # MSMARCO
│   └─ miracl/                           # MIRACL

```

```
|   ├── mkqa/                # MKQA  ██
|   ├── mldr/                # MLDR  ██
|   ├── air_bench/           # AIR-Bench  ██
|   ├── bright/              # BRIGHT  ██
|   ├── custom/              #  █████
|   └── __init__.py
└── utils/                   #  █████
    ├── transformers_compat.py # Transformers  █████
    └── __init__.py
__init__.py
```

3.1 █████

██	██
abc/	████████████████████
inference/	████████████████████
finetune/	██████████
evaluation/	██████████
utils/	██████████

4. ██████████

4.1 █████

FlagEmbedding ██████████

BGE ██████████

- **bge-m3**████████████████████
- **bge-large/ base/ small-en-v1.5**███ v1.5 ██
- **bge-large/ base/ small-zh-v1.5**███ v1.5 ██
- **bge-en-icl**████████████████████
- **bge-multilingual-gemma2**███ Gemma2 █████
- **bge-code-v1**██████

MARCO  -
 - **AIR-Bench**  -

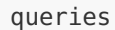
5.

5.1

 FlagAutoModel 

```
from FlagEmbedding import FlagAutoModel

#  BGE-M3 
model = FlagAutoModel.from_finetuned(
    'BAAI/bge-m3',
    normalize_embeddings=True,
    use_fp16=True
)

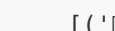
# 
queries = ['', '']
query_embeddings = model.encode_queries(queries)

# 
corpus = [
    '',
    '']
corpus_embeddings = model.encode_corpus(corpus)
```

5.2

```
from FlagEmbedding import FlagAutoReranker

# 
reranker = FlagAutoReranker.from_finetuned('BAAI/bge-reranker-large')

# 
scores = reranker.compute_score(
    [
        ('', ''),
        ('', '')
    ]
)
```

5.3

```
from FlagEmbedding import FlagAutoModel

# 使用 GPU 推理
model = FlagAutoModel.from_finetuned(
    'BAAI/bge-large-en-v1.5',
    devices=[0, 1, 2, 3] # 使用 4 个 GPU
)

# 推理
embeddings = model.encode(large_corpus, batch_size=1024)
```

6.

1. 
2.  FlagAutoModel  FlagAutoReranker 
3.  CUDA  NPU  MUSA  MPS  CPU 
4.  GPU
5. 
6.  BGE  E5  GTE  Qwen3 
7. 

7.

-   
-   
-   
-   

02 abc layer

FlagEmbedding 0000 (ABC Layer) 000

000000

Mermaid Diagram

00000000

00	00	00000	00000
AbsEmbedder	00 00 00	encode/ encode_queries/ encode_corpus	abc/inference/ AbsEmbedder.py
AbsReranker	00 00 00 0	compute_score	abc/inference/ AbsReranker.py
AbsEmbedderModel	00 00 00	forward/loss 00	abc/finetune/ embedder/ AbsModeling.py
AbsRerankerModel	00 00 00 0	forward	abc/finetune/reranker/ AbsModeling.py
AbsEvalRunner	00 00 0	run	abc/evaluation/ runner.py
AbsEvaluator	00 00 0	call	abc/evaluation/ evaluator.py

00

- 1. 00
- 2. 000000
- 2.1 AbsEmbedder
- 2.2 AbsReranker
- 3. 000000
- 3.1 Embedder 00000

- 3.2 Reranker [] [] [] []
- 4. [] [] [] []
- 4.1 AbsEvalRunner
- 4.2 AbsEvaluator
- 4.3 AbsEvalDataLoader
- 4.4 [] [] [] []
- 5. [] [] [] [] [] [] [] []

1.

FlagEmbedding  (ABC Layer) 

- $\mathbf{Q}, \mathbf{K}, \mathbf{V}$: $\mathbf{Q}, \mathbf{K}$ embedding $\rightarrow$ reranking $\mathbf{Q}, \mathbf{K}$
- $\mathbf{Q}, \mathbf{K}, \mathbf{V}$: $\mathbf{Q}, \mathbf{K}, \mathbf{V}$
- $\mathbf{Q}, \mathbf{K}, \mathbf{V}$: $\mathbf{Q}, \mathbf{K}, \mathbf{V}$

目录

```

abc/
├── inference/                # 推理
│   ├── AbsEmbedder.py
│   └── AbsReranker.py
├── finetune/                # 微调
│   ├── embedder/
│   │   ├── AbsModeling.py
│   │   ├── AbsDataset.py
│   │   ├── AbsTrainer.py
│   │   ├── AbsRunner.py
│   │   └── AbsArguments.py
│   └── reranker/
│       ├── AbsModeling.py
│       ├── AbsDataset.py
│       ├── AbsTrainer.py
│       ├── AbsRunner.py
│       └── AbsArguments.py
└── evaluation/              # 评估
    ├── runner.py
    ├── evaluator.py
    ├── data_loader.py
    ├── searcher.py
    ├── arguments.py
    └── utils.py
  
```

2. 目录

2.1 AbsEmbedder

文件: [abc/inference/AbsEmbedder.py](#)

AbsEmbedder

□□□□

```

class AbsEmbedder(ABC):
    def __init__(
        self,
        model_name_or_path: str,
        normalize_embeddings: bool = True,
        use_fp16: bool = True,
        use_bf16: bool = False,
        query_instruction_for_retrieval: Optional[str] = None,
        query_instruction_format: str = "{}{}",
        devices: Optional[Union[str, int, List[str], List[int]]] = None,
        batch_size: int = 256,
        query_max_length: int = 512,
        passage_max_length: int = 512,
        convert_to_numpy: bool = True,
        truncate_dim: Optional[int] = None,
        **kwargs: Any,
    ):

```

□□□□: - normalize_embeddings : □□□□□□□□ - use_fp16 / use_bf16 : □□□□□□□□ - devices : □□
□□□□GPU/TPU/NPU/Musa□□□□□□ - truncate_dim : □□ Matryoshka Representation
Learning (MRL) □□□□□

□□□□□□□**1.** □□□□□□□

```

def encode_queries(
    self,
    queries: Union[List[str], str],
    batch_size: Optional[int] = None,
    max_length: Optional[int] = None,
    convert_to_numpy: Optional[bool] = None,
    **kwargs: Any
)

```

□□□□□□□□□□□□□□□□ query_instruction_for_retrieval □

```
def encode_corpus(
    self,
    corpus: Union[List[str], str],
    batch_size: Optional[int] = None,
    max_length: Optional[int] = None,
    convert_to_numpy: Optional[bool] = None,
    **kwargs: Any
)
```

 passage_instruction_for_retrieval 

```
def encode(
    self,
    sentences: Union[List[str], str],
    batch_size: Optional[int] = None,
    max_length: Optional[int] = None,
    convert_to_numpy: Optional[bool] = None,
    instruction: Optional[str] = None,
    instruction_format: Optional[str] = None,
    **kwargs: Any
)
```

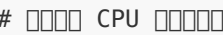
 encode_queries  encode_corpus 

2.

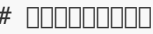
```
@abstractmethod
def encode_single_device(
    self,
    sentences: Union[List[str], str],
    batch_size: int = 256,
    max_length: int = 512,
    convert_to_numpy: bool = True,
    device: Optional[str] = None,
    **kwargs: Any,
):
    """"""
    pass
```

AbsEmbedder 

1.  ([start_multi_process_pool](#))
2.  spawn 
3. 
4. 

```
def start_multi_process_pool(
    self,
    process_target_func: Any,
) -> Dict[Literal["input", "output", "processes"], Any]:
    #  CPU 
    self.model.to("cpu")
    self.model.share_memory()

    #  spawn 
    ctx = mp.get_context("spawn")
    input_queue = ctx.Queue()
    output_queue = ctx.Queue()
    processes = []

    # 
    for device_id in self.target_devices:
        p = ctx.Process(
            target=process_target_func,
            args=(device_id, self, input_queue, output_queue),
            daemon=True,
        )
        p.start()
        processes.append(p)
```

1.  ([_encode_multi_process_worker](#))
2. 
3.  encode_single_device 
4. 

5.  (`encode_multi_process`)

6. 

7. 

8. 

```
def encode_multi_process(
    self,
    sentences: List[str],
    pool: Dict[Literal["input", "output", "processes"], Any],
    **kwargs
):
    # 
    chunk_size = math.ceil(len(sentences) / len(pool["processes"]))

    # 
    input_queue = pool["input"]
    last_chunk_id = 0
    chunk = []
    for sentence in sentences:
        chunk.append(sentence)
        if len(chunk) >= chunk_size:
            input_queue.put([last_chunk_id, chunk, kwargs])
            last_chunk_id += 1
            chunk = []

    # ... 

    # 
    results_list = sorted(
        [output_queue.get() for _ in trange(last_chunk_id)],
        key=lambda x: x[0],
    )
    # 
    return self._concatenate_results_from_multi_process([result[1] for result in results_list])
```

1. 

2. `stop_self_pool` :  GPU 

3. `__del__` : 

□ □ □ □ □ □

get_target_devices

```
@staticmethod
def get_target_devices(devices: Union[str, int, List[str], List[int]]) -> List[str]:
    if devices is None:
        # 默认设备
        if torch.cuda.is_available():
            return [f"cuda:{i}" for i in range(torch.cuda.device_count())]
        elif is_torch_npu_available():
            return [f"npu:{i}" for i in range(torch.npu.device_count())]
        elif hasattr(torch, "musa") and torch.musa.is_available():
            return [f"musa:{i}" for i in range(torch.musa.device_count())]
        elif torch.backends.mps.is_available():
            # ... MPS 设备
        else:
            return ["cpu"]
    # ... 其他设备类型
```

□□□□□□ - NVIDIA CUDA GPU - Huawei NPU - Moore Threads Musa - Apple MPS - CPU

□ □ □ □

1. `get_detailed_instruct` (python @staticmethod def
get_detailed_instruct(instruction_format: str, instruction: str, sentence: str):
""" if "\\n" in instruction_format: instruction_format =
instruction_format.replace("\\n", "\n") return
instruction_format.format(instruction, sentence)
2. `_concatenate_results_from_multi_process`
3. `torch.Tensor` `numpy.ndarray`
4. **NumPy** `_convert_to_numpy`
5. `bf16` `NumPy` `bf16`
6. `bf16` $\rightarrow$ `float32`
7. `_truncate_embeddings`
8. `MRL`

2.2 AbsReranker

: [abc/inference/AbsReranker.py](#)

AbsReranker -



```
class AbsReranker(ABC):
    def __init__(
        self,
        model_name_or_path: str,
        use_fp16: bool = False,
        query_instruction_for_rerank: Optional[str] = None,
        query_instruction_format: str = "{}{}",
        passage_instruction_for_rerank: Optional[str] = None,
        passage_instruction_format: str = "{}{}",
        devices: Optional[Union[str, int, List[str], List[int]]] = None,
        batch_size: int = 128,
        query_max_length: Optional[int] = None,
        max_length: int = 512,
        normalize: bool = False,
        **kwargs: Any,
    ):
```

: -  -  AbsEmbedder  - 

□□□□**1.** □□□□

```

def compute_score(
    self,
    sentence_pairs: Union[List[Tuple[str, str]], Tuple[str, str]],
    **kwargs
):
    """□□□□□□□□□□"""
    if isinstance(sentence_pairs[0], str):
        sentence_pairs = [sentence_pairs]

    # □□□□□□
    sentence_pairs = self.get_detailed_inputs(sentence_pairs)

    # □□□□□□□□□□
    if isinstance(sentence_pairs, str) or len(self.target_devices) == 1:
        return self.compute_score_single_gpu(...)
    else:
        # □□□□□□
        if self.pool is None:
            self.pool = self.start_multi_process_pool()
        return self.encode_multi_process(...)

```

2. □□□□

```

@abstractmethod
def compute_score_single_gpu(
    self,
    sentence_pairs: Union[List[Tuple[str, str]], Tuple[str, str]],
    batch_size: int = 256,
    query_max_length: Optional[int] = None,
    max_length: int = 512,
    normalize: bool = False,
    device: Optional[str] = None,
    **kwargs: Any,
):
    """□□□□□□□□□□□□□□"""
    pass

```

□ □ □ □ □

get_detailed_inputs 

```
def get_detailed_inputs(self, sentence_pairs: Union[str, List[str]]):
    """get_detailed_inputs"""
    if isinstance(sentence_pairs, str):
        sentence_pairs = [sentence_pairs]

    if self.query_instruction_for_rerank is not None:
        if self.passage_instruction_for_rerank is None:
            return [
                [
                    self.get_detailed_instruct(self.query_instruction_format,
                                                self.query_instruction_for_rerank,
                                                sentence_pair[0]),
                    sentence_pair[1]
                ] for sentence_pair in sentence_pairs
            ]
        else:
            return [
                [
                    self.get_detailed_instruct(self.query_instruction_format,
                                                self.query_instruction_for_rerank,
                                                sentence_pair[0]),
                    self.get_detailed_instruct(self.passage_instruction_format,
                                                self.passage_instruction_for_rerank,
                                                sentence_pair[1])
                ] for sentence_pair in sentence_pairs
            ]

    # ... get_detailed_inputs
```

□ □ □ □ □

```

AbsEmbedder [ ] - start_multi_process_pool -
encode multi process worker - encode multi process

```

3.

3.1 Embedder

3.1.1 AbsEmbedderModel ()

: [abc/finetune/embedder/AbsModeling.py](#)





```
@dataclass
class EmbedderOutput(ModelOutput):
    q_reps: Optional[Tensor] = None
    p_reps: Optional[Tensor] = None
    loss: Optional[Tensor] = None
    scores: Optional[Tensor] = None
```

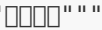


```
class AbsEmbedderModel(ABC, nn.Module):
    def __init__(
        self,
        base_model,
        tokenizer: PreTrainedTokenizer = None,
        negatives_cross_device: bool = False,
        temperature: float = 1.0,
        sub_batch_size: int = -1,
        kd_loss_type: str = 'kl_div',
        use_mrl: bool = False,
        mrl_dims: List[int] = [],
    ):
```

: - negatives_cross_device:  - temperature:  -
kd_loss_type:  (kl_div  m3_kd_loss) - use_mrl:  Matryoshka
Representation Learning - mrl_dims: MRL 



```
@abstractmethod
def encode(self, features):
    """"""
    pass

@abstractmethod
def compute_loss(self, scores, target):
    """"""
    pass

@abstractmethod
def compute_score(self, q_reps, p_reps):
    """"""
    pass

@abstractmethod
def save(self, output_dir: str):
    """"""
    pass
```



1. 旗子 (_compute_no_in_batch_neg_loss)

```

def _compute_no_in_batch_neg_loss(self, q_reps, p_reps, teacher_targets=None, ...):
    """旗子"""
    group_size = p_reps.size(0) // q_reps.size(0)

    # 旗子
    local_scores = self.compute_local_score(q_reps, p_reps, ...)

    if teacher_targets is not None:
        # 旗子
        loss = self.distill_loss(self.kd_loss_type, teacher_targets, local_scores, group_size=group_size)
        if self.kd_loss_type == "kl_div":
            # 旗子
            local_targets = torch.zeros(local_scores.size(0), ...)
            loss += self.compute_loss(local_scores, local_targets)
    else:
        local_targets = torch.zeros(local_scores.size(0), ...)
        loss = self.compute_loss(local_scores, local_targets)

    return local_scores, loss

```

2. 计算损失 (compute_in_batch_neg_loss)

```
def _compute_in_batch_neg_loss(self, q_reps, p_reps, teacher_targets=None, ...):
    """
    group_size = p_reps.size(0) // q_reps.size(0)

    #
    if compute_score_func is None:
        scores = self.compute_score(q_reps, p_reps) # (batch_size, batch_size * group_size)
    else:
        scores = compute_score_func(q_reps, p_reps, ...)

    if teacher_targets is not None:
        if self.kd_loss_type == "kl_div":
            #
            student_scores = self.get_local_score(q_reps, p_reps, scores)
            loss = self.distill_loss(...)
            #
            idxs = torch.arange(q_reps.size(0), ...)
            targets = idxs * group_size
            loss += self.compute_loss(scores, targets)
        elif self.kd_loss_type == "m3_kd_loss":
            # M3
            loss = self.distill_loss(...)
        else:
            idxs = torch.arange(q_reps.size(0), ...)
            targets = idxs * group_size #
            loss = self.compute_loss(scores, targets)

    return scores, loss
```

3. `compute_cross_device_neg_loss`

```
def _compute_cross_device_neg_loss(self, q_reps, p_reps, teacher_targets=None, ...):
    """compute_cross_device_neg_loss"""
    group_size = p_reps.size(0) // q_reps.size(0)

    # gather
    cross_q_reps = self._dist_gather_tensor(q_reps) # (world_size * batch_size, dim)
    cross_p_reps = self._dist_gather_tensor(p_reps) # (world_size * batch_size * group_size, dim)

    # compute
    if compute_score_func is None:
        cross_scores = self.compute_score(cross_q_reps, cross_p_reps)
    else:
        cross_scores = compute_score_func(cross_q_reps, cross_p_reps, ...)

    # ... compute_loss
    return cross_scores, loss
```

`compute_loss``_dist_gather_tensor` 

```

def _dist_gather_tensor(self, t: Optional[torch.Tensor]):
    """□□□□□□□□□□"""
    if t is None:
        return None
    t = t.contiguous()

    # □□□□□□□□
    all_tensors = [torch.empty_like(t) for _ in range(self.world_size)]

    # □□□□□□□□□□
    dist.all_gather(all_tensors, t)

    # □□□□□□□□□□□□□□□□
    all_tensors[self.process_rank] = t

    # □□
    all_tensors = torch.cat(all_tensors, dim=0)

    return all_tensors

```

□□□□□□distill_loss □□□□□□□

```

@staticmethod
def distill_loss(kd_loss_type, teacher_targets, student_scores, group_size=None):
    if kd_loss_type == 'kl_div':
        # KL
        return -torch.mean(
            torch.sum(torch.log_softmax(student_scores, dim=-1) * teacher_targets, dim=-1)
        )
    elif kd_loss_type == 'm3_kd_loss':
        # BGE-M3
        labels = torch.arange(student_scores.size(0), ...)
        labels = labels * group_size
        loss = 0
        mask = torch.zeros_like(student_scores)
        for i in range(group_size):
            temp_target = labels + i
            temp_scores = student_scores + mask
            temp_loss = F.cross_entropy(temp_scores, temp_target, reduction="none")
            loss += torch.mean(teacher_targets[:, i] * temp_loss)
        #
        mask = torch.scatter(mask, dim=-1, index=temp_target.unsqueeze(-1),
                             value=torch.finfo(student_scores.dtype).min)
    return loss

```

[] [] [] []

forward [] [] [] [] [] [] [] [] []

```

def forward(
    self,
    queries: Union[Dict[str, Tensor], List[Dict[str, Tensor]]] = None,
    passages: Union[Dict[str, Tensor], List[Dict[str, Tensor]]] = None,
    teacher_scores: Union[None, List[float]] = None,
    no_in_batch_neg_flag: bool = False,
):
    # [ ] [ ] [ ] [ ] [ ] [ ]
    q_reps = self.encode(queries)
    p_reps = self.encode(passages)

    if self.training:
        # [ ] [ ] [ ] [ ] [ ]
        if teacher_scores is not None:
            teacher_scores = torch.tensor(teacher_scores, ...)
            teacher_scores = teacher_scores.view(...)
            teacher_targets = F.softmax(teacher_scores, dim=-1)
        else:
            teacher_targets = None

        # [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
        if no_in_batch_neg_flag:
            compute_loss_func = self._compute_no_in_batch_neg_loss
        else:
            if self.negatives_cross_device:
                compute_loss_func = self._compute_cross_device_neg_loss
            else:
                compute_loss_func = self._compute_in_batch_neg_loss

        # MRL [ ] [ ]
        if self.use_mrl:
            all_loss = torch.tensor(0.0, ...)
            # [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
            for dim_q_reps, dim_p_reps in zip(q_reps, p_reps):
                _, mrl_loss = compute_loss_func(dim_q_reps, dim_p_reps, teacher_targets=teacher_targets)
                all_loss += mrl_loss
            loss = all_loss / len(self.mrl_dims)
        else:
            scores, loss = compute_loss_func(q_reps, p_reps, teacher_targets=teacher_targets)
    else:

```

```
loss = None
```

```
return EmbedderOutput(loss=loss)
```

□□□□

1. `get_local_score` python

```
def get_local_score(self, q_reps, p_reps, all_scores): """ group_size = p_reps.size(0) // q_reps.size(0) indices = torch.arange(0, q_reps.size(0), ...) * group_size specific_scores = [] for i in range(group_size): specific_scores.append( all_scores[torch.arange(q_reps.size(0), ...), indices + i] ) return torch.stack(specific_scores, dim=1).view(q_reps.size(0), -1)
```

3.1.2 AbsEmbedderTrainDataset (□□□□)

```

[ ] [ ] [ ] [ ]: abc/finetune/embedder/AbsDataset.py

```

□□□□□□

```
class AbsEmbedderTrainDataset(Dataset):
    def __init__(
        self,
        args: AbsEmbedderDataArguments,
        tokenizer: PreTrainedTokenizer
    ):
        self.args = args
        self.tokenizer = tokenizer
        self.shuffle_ratio = args.shuffle_ratio

        # 遍历数据集
        train_datasets = []
        for data_dir in args.train_data:
            # ... 加载 json/jsonl 文件
            temp_dataset = self._load_dataset(data_dir)
            train_datasets.append(temp_dataset)

        self.dataset = datasets.concatenate_datasets(train_datasets)
```

□□□□ (□□□□):

```
{
  "query": "????",
  "pos": ["????1", "????2"],
  "neg": ["????1", "????2"],
  "pos_scores": [0.9, 0.8], // ?????
  "neg_scores": [0.3, 0.2]  // ??
}
```

□□□□ (**getitem**)

```

def __getitem__(self, item):
    data = self.dataset[item]
    train_group_size = self.args.train_group_size

    query = data['query']
    # □□□□□□
    if self.args.query_instruction_for_retrieval is not None:
        query = self.args.query_instruction_format.format(...)

    passages = []
    teacher_scores = []

    # □□□□□□□□□□
    pos_idx = random.choice(list(range(len(data['pos']))))
    passages.append(self._shuffle_text(data['pos'][pos_idx]))

    # □□□□□□□□□□□□□□
    neg_all_idx = list(range(len(data['neg'])))
    if len(data['neg']) < train_group_size - 1:
        num = math.ceil((train_group_size - 1) / len(data['neg']))
        neg_idxes = random.sample(neg_all_idx * num, train_group_size - 1)
    else:
        neg_idxes = random.sample(neg_all_idx, self.args.train_group_size - 1)

    for neg_idx in neg_idxes:
        passages.append(data['neg'][neg_idx])

    # □□□□□□
    if self.args.knowledge_distillation:
        teacher_scores.append(data['pos_scores'][pos_idx])
        for neg_idx in neg_idxes:
            teacher_scores.append(data['neg_scores'][neg_idx])

    # □□□□□□
    if self.args.passage_instruction_for_retrieval is not None:
        passages = [self.args.passage_instruction_format.format(...) for p in passages]

    return query, passages, teacher_scores

```

[_shuffle_text](#) 

```
def _shuffle_text(self, text):
    """"""
    if self.shuffle_ratio > 0 and len(text) > 100 and random.random() < self.shuffle_ratio:
        split_text = []
        chunk_size = len(text)//3 + 1
        for i in range(0, len(text), chunk_size):
            split_text.append(text[i:i+chunk_size])
        random.shuffle(split_text)
        return " ".join(split_text)
    else:
        return text
```

 (**AbsEmbedderSameDatasetTrainDataset**) batch 

```

class AbsEmbedderSameDatasetTrainDataset(AbsEmbedderTrainDataset):
    def __init__(
        self,
        args: AbsEmbedderDataArguments,
        default_batch_size: int,
        seed: int,
        tokenizer: PreTrainedTokenizer,
        process_index: int = 0,
        num_processes: int = 1
    ):
        # ... □□□□□
        # □□□□□□□□□□ batch size
        batch_size_idxes.append(self._get_file_batch_size(temp_dataset, default_batch_size))
        # □□ no_in_batch_neg □□
        no_in_batch_neg_flags.append(no_in_batch_neg_flag)

        # □□□ batch □□
        self.refresh_epoch()

    def refresh_epoch(self):
        """□□ epoch □□□□□□□□ batch"""
        self.deterministic_generator.shuffle(self.datasets_inxs)
        batch_datas = []
        for dataset_inx in self.datasets_inxs:
            self.deterministic_generator.shuffle(self.each_data_idxes[dataset_inx])
            cur_batch_size = self.batch_size_idxes[dataset_inx] * self.num_processes
            # ... □□ batch
        self.deterministic_generator.shuffle(batch_datas)
        self.batch_datas = batch_datas

```

□□□□: - □□□□□□□□□□ batch size - □□□□□□□□□□□□□□□□ - □□

EmbedderTrainerCallbackForDataRefresh □□□□□ epoch □□□□□

DataCollator

```

@dataclass
class AbsEmbedderCollator(DataCollatorWithPadding):
    query_max_len: int = 32
    passage_max_len: int = 128
    sub_batch_size: int = -1

    def __call__(self, features):
        queries = [f[0] for f in features]
        passages = [f[1] for f in features]
        teacher_scores = [f[2] for f in features]

        # []
        if isinstance(queries[0], list):
            queries = sum(queries, [])
        if isinstance(passages[0], list):
            passages = sum(passages, [])

        # tokenize
        queries_inputs = self.tokenizer(queries, truncation=True, max_length=self.query_max_len)
        passages_inputs = self.tokenizer(passages, truncation=True, max_length=self.passage_max_len)

        # []
        if self.sub_batch_size is None or self.sub_batch_size <= 0:
            q_collated = self.tokenizer.pad(queries_inputs, ...)
            d_collated = self.tokenizer.pad(passages_inputs, ...)
        else:
            # []
            q_collated = []
            for i in range(0, len(queries_inputs['attention_mask']), self.sub_batch_size):
                # ... pad []
            d_collated = []
            # ... []

        return {
            "queries": q_collated,
            "passages": d_collated,
            "teacher_scores": teacher_scores,
            "no_in_batch_neg_flag": False
        }

```

3.1.3 AbsEmbedderTrainer (接口)

位置: [abc/finetune/embedder/AbsTrainer.py](#)

```
class AbsEmbedderTrainer(ABC, Trainer):
    @abstractmethod
    def _save(self, output_dir: Optional[str] = None, state_dict=None):
        """保存模型"""
        pass

    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        """计算损失"""
        outputs = model(**inputs)
        loss = outputs.loss
        return (loss, outputs) if return_outputs else loss
```



3.1.4 AbsEmbedderRunner (接口)

位置: [abc/finetune/embedder/AbsRunner.py](#)



```

class AbsEmbedderRunner(ABC):
    def __init__(
        self,
        model_args: AbsEmbedderModelArguments,
        data_args: AbsEmbedderDataArguments,
        training_args: AbsEmbedderTrainingArguments
    ):
        # ... 
        self.tokenizer, self.model = self.load_tokenizer_and_model()
        self.train_dataset = self.load_train_dataset()
        self.data_collator = self.load_data_collator()
        self.trainer = self.load_trainer()

    @abstractmethod
    def load_tokenizer_and_model(self) -> Tuple[PreTrainedTokenizer, AbsEmbedderModel]:
        pass

    @abstractmethod
    def load_trainer(self) -> AbsEmbedderTrainer:
        pass

    def load_train_dataset(self) -> AbsEmbedderTrainDataset:
        """"""
        if self.data_args.same_dataset_within_batch:
            return AbsEmbedderSameDatasetTrainDataset(...)
        else:
            return AbsEmbedderTrainDataset(...)

    def load_data_collator(self) -> AbsEmbedderCollator:
        """ collator"""
        if self.data_args.same_dataset_within_batch:
            return AbsEmbedderSameDatasetCollator(...)
        else:
            return AbsEmbedderCollator(...)

    def run(self):
        """"""
        Path(self.training_args.output_dir).mkdir(parents=True, exist_ok=True)
        self.trainer.train(resume_from_checkpoint=self.training_args.resume_from_checkpoint)
        self.trainer.save_model()

```

3.1.5 AbsEmbedderArguments ([] [] [] [])

[] [] [] []: [abc/finetune/embedder/AbsArguments.py](#)

[] [] [] [] [] [] [] []

1. **AbsEmbedderModelArguments:** [] [] [] []

2. `model_name_or_path`

3. `config_name`

4. `tokenizer_name`

5. []

6. **AbsEmbedderDataArguments:** [] [] [] []

7. `train_data` : [] [] [] [] [] [] [] []

8. `train_group_size` : [] [] [] [] [] [] [] [] 1 [] + (n-1) []

9. `query_max_len` / `passage_max_len`

10. `knowledge_distillation` : [] [] [] [] KD

11. `same_dataset_within_batch` : [] [] [] [] [] batch

12. `shuffle_ratio` : [] [] [] [] [] []

13. []

14. **AbsEmbedderTrainingArguments:** [] [] [] [] [] [] transformers.TrainingArguments []

15. `negatives_cross_device` : [] [] [] [] [] []

16. `temperature` : [] [] [] []

17. `sentence_pooling_method` : [] [] [] (`cls` / `mean` / `last_token`)

18. `normalize_embeddings` : [] [] [] [] []

19. `kd_loss_type` : [] [] [] [] [] []

20. `use_mrl` / `mrl_dims` : MRL [] []

21. []

3.2 Reranker [] [] [] [] []

Reranker [] [] [] [] [] [] Embedder [] [] [] [] [] [] [] []

3.2.1 AbsRerankerModel

: [abc/finetune/reranker/AbsModeling.py](#)

```

@dataclass
class RerankerOutput(ModelOutput):
    loss: Optional[Tensor] = None
    scores: Optional[Tensor] = None

class AbsRerankerModel(ABC, nn.Module):
    def __init__(
        self,
        base_model,
        tokenizer: PreTrainedTokenizer = None,
        train_batch_size: int = 4,
    ):
        nn.Module.__init__(self)
        self.model = base_model
        self.tokenizer = tokenizer
        self.cross_entropy = nn.CrossEntropyLoss(reduction='mean')

        #  pad token
        if self.model.config.pad_token_id is None:
            self.model.config.pad_token_id = self.tokenizer.pad_token_id
        self.config = self.model.config

        self.train_batch_size = train_batch_size

        #  decoder-only  Yes 
        self.yes_loc = self.tokenizer('Yes', add_special_tokens=False)['input_ids'][-1]

    @abstractmethod
    def encode(self, features):
        pass

    def forward(self, pair: Union[Dict[str, Tensor], List[Dict[str, Tensor]]] = None,
                teacher_scores: Optional[Tensor] = None):
        ranker_logits = self.encode(pair)

        if self.training:
            #  (batch_size, group_size)
            grouped_logits = ranker_logits.view(self.train_batch_size, -1)
            target = torch.zeros(self.train_batch_size, ..., dtype=torch.long)

```

□□□□: - □ Embedder □□□□□□□□ - □□□□□□□□□□ - □□□□□

- **AbsRerankerTrainDataset:** [abc/finetune/reranker/AbsDataset.py](#)
- `Embedder` query-passage
- `AbsLLMRerankerTrainDataset` decoder-only
- **AbsRerankerTrainer:** [abc/finetune/reranker/AbsTrainer.py](#)
- **AbsRerankerRunner:** [abc/finetune/reranker/AbsRunner.py](#)
- **AbsRerankerArguments:** [abc/finetune/reranker/AbsArguments.py](#)
- `model_type` encoder/decoder
- `sep_token` LLM

4. █████

4.1 AbsEvalRunner

████: [abc/evaluation/runner.py](#)

████████████████████

```

class AbsEvalRunner:
    def __init__(
        self,
        eval_args: AbsEvalArgs,
        model_args: AbsEvalModelArgs,
    ):
        self.eval_args = eval_args
        self.model_args = model_args

        # 加载
        self.retriever, self.reranker = self.load_retriever_and_reranker()
        self.data_loader = self.load_data_loader()
        self.evaluator = self.load_evaluator()

    @staticmethod
    def get_models(model_args: AbsEvalModelArgs) -> Tuple[AbsEmbedder, Union[AbsReranker, None]]:
        """加载 Embedder 和 Reranker"""
        embedder = FlagAutoModel.from_finetuned(...)
        embedder.model.config._name_or_path = model_args.embedder_name_or_path

        reranker = None
        if model_args.reranker_name_or_path is not None:
            reranker = FlagAutoReranker.from_finetuned(...)
            reranker.model.config._name_or_path = model_args.reranker_name_or_path

        return embedder, reranker

    def run(self):
        """运行"""
        if self.eval_args.dataset_names is None:
            dataset_names = self.data_loader.available_dataset_names()
        else:
            dataset_names = self.data_loader.check_dataset_names(self.eval_args.dataset_names)

        if len(dataset_names) == 0:
            # 没有数据集
            self.evaluator(
                splits=self.eval_args.splits,
                search_results_save_dir=self.eval_args.output_dir,
                retriever=self.retriever,

```

```

        reranker=self.reranker,
        corpus_embd_save_dir=self.eval_args.corpus_embd_save_dir,
        ignore_identical_ids=self.eval_args.ignore_identical_ids,
        k_values=self.eval_args.k_values
    )
else:
    # 数据集
    for dataset_name in dataset_names:
        self.evaluator(
            splits=self.eval_args.splits,
            search_results_save_dir=self.eval_args.output_dir,
            retriever=self.retriever,
            reranker=self.reranker,
            corpus_embd_save_dir=self.eval_args.corpus_embd_save_dir,
            ignore_identical_ids=self.eval_args.ignore_identical_ids,
            k_values=self.eval_args.k_values,
            dataset_name=dataset_name,
        )

    # 评估
    self.evaluate_metrics(
        search_results_save_dir=self.eval_args.output_dir,
        output_method=self.eval_args.eval_output_method,
        output_path=self.eval_args.eval_output_path,
        metrics=self.eval_args.eval_metrics
    )

```

4.2 AbsEvaluator

□□□□: [abc/evaluation/evaluator.py](#)

```

class AbsEvaluator:
    def __init__(
        self,
        eval_name: str,
        data_loader: AbsEvalDataLoader,
        overwrite: bool = False,
    ):
        self.eval_name = eval_name
        self.data_loader = data_loader
        self.overwrite = overwrite

    def __call__(
        self,
        splits: Union[str, List[str]],
        search_results_save_dir: str,
        retriever: EvalRetriever,
        reranker: Optional[EvalReranker] = None,
        corpus_embd_save_dir: Optional[str] = None,
        ignore_identical_ids: bool = False,
        k_values: List[int] = [1, 3, 5, 10, 100, 1000],
        dataset_name: Optional[str] = None,
        **kwargs,
    ):
        # 1.  splits
        checked_splits = self.data_loader.check_splits(splits, dataset_name=dataset_name)

        # 2. 
        no_reranker_search_results_save_dir = os.path.join(
            search_results_save_dir, str(retriever), "NoReranker"
        )
        os.makedirs(no_reranker_search_results_save_dir, exist_ok=True)

        flag = False # 
        for split in checked_splits:
            save_path = os.path.join(no_reranker_search_results_save_dir, save_name)
            if not os.path.exists(save_path) or self.overwrite:
                flag = True
                break

        no_reranker_search_results_dict = {}

```

```

if flag:
    # [ ] [ ] [ ] [ ]
    corpus = self.data_loader.load_corpus(dataset_name=dataset_name)
    queries_dict = {split: self.data_loader.load_queries(...) for split in checked_splits}
    all_queries = {k: v for d in queries_dict.values() for k, v in d.items()}

    all_no_reranker_search_results = retriever(
        corpus=corpus,
        queries=all_queries,
        corpus_embd_save_dir=corpus_embd_save_dir,
        ignore_identical_ids=ignore_identical_ids,
        **kwargs,
    )

    # [ ] [ ] [ ] [ ]
    for split in checked_splits:
        no_reranker_search_results_dict[split] = {
            qid: all_no_reranker_search_results[qid]
            for qid in queries_dict[split]
        }
        self.save_search_results(
            eval_name=self.eval_name,
            model_name=str(retriever),
            reranker_name="NoReranker",
            search_results=no_reranker_search_results_dict[split],
            output_path=os.path.join(no_reranker_search_results_save_dir, save_name),
            split=split,
            dataset_name=dataset_name,
        )
else:
    # [ ] [ ] [ ] [ ] [ ] [ ]
    for split in checked_splits:
        data_info, search_results = self.load_search_results(...)
        self.check_data_info(data_info, ...)
        no_reranker_search_results_dict[split] = search_results

    # [ ] [ ] [ ] [ ]
    retriever.stop_multi_process_pool()

    # 3. [ ] [ ] [ ] [ ] [ ] [ ] [ ]
    eval_results_save_path = os.path.join(no_reranker_search_results_save_dir, 'EVAL', 'eval_

```

```

if not os.path.exists(eval_results_save_path) or self.overwrite or flag:
    retriever_eval_results = self.evaluate_results(no_reranker_search_results_save_dir, k)
    self.output_eval_results_to_json(retriever_eval_results, eval_results_save_path)

# 4. 保存 reranker
if reranker is not None:
    # ...
    reranker_search_results_save_dir = os.path.join(
        search_results_save_dir, str(retriever), str(reranker)
    )
    # ...
    # ...
    # ...

```

--	--	--	--

1. **save_search_results / load_search_results:** JSON 
2. **compute_metrics:** 
3. **evaluate_results:** 
4. **output_eval_results_to_json / output_eval_results_to_markdown:** 

4.3 AbsEvalDataLoader

```

0000: abc/evaluation/data_loader.py

```

--	--	--	--	--	--	--	--	--

```

class AbsEvalDataLoader(ABC):
    def __init__(
        self,
        eval_name: str,
        dataset_dir: Optional[str] = None,
        cache_dir: Optional[str] = None,
        token: Optional[str] = None,
        force_redownload: bool = False
    ):
        self.eval_name = eval_name
        self.dataset_dir = dataset_dir
        self.cache_dir = os.path.join(cache_dir, eval_name) if cache_dir else ...
        self.force_redownload = force_redownload

    def available_dataset_names(self) -> List[str]:
        return []

    @abstractmethod
    def available_splits(self, dataset_name: Optional[str] = None) -> List[str]:
        pass

    def load_corpus(self, dataset_name: Optional[str] = None):
        """"""
        if self.dataset_dir is not None:
            return self._load_local_corpus(...)
        else:
            return self._load_remote_corpus(...)

    def load_queries(self, dataset_name: Optional[str] = None, split: str = 'test'):
        """"""
        # ... 

    def load_qrels(self, dataset_name: Optional[str] = None, split: str = 'test'):
        """ qrels ()"""
        # ... 

    # 
    def _load_remote_corpus(self, ...):
        raise NotImplementedError

    def _load_remote_queries(self, ...):

```

```

        raise NotImplementedError
    def _load_remote_qrels(self, ...):
        raise NotImplementedError

```

```

□□□□□□□ (□□): - corpus.jsonl : {"id": "...", "title": "...", "text": "..."} - {split}
_queries.jsonl : {"id": "...", "text": "..."} - {split}_qrels.jsonl : {"qid": "...",
"docid": "...", "relevance": 1}

```

4.4 □□□□□

□□□□: <abc/evaluation/searcher.py>

EvalRetriever (□□□□□)

```

class EvalRetriever(ABC):
    def __init__(self, embedder: AbsEmbedder, search_top_k: int = 1000, overwrite: bool = False):
        self.embedder = embedder
        self.search_top_k = search_top_k
        self.overwrite = overwrite

    def __str__(self) -> str:
        return os.path.basename(self.embedder.model.config._name_or_path)

    def stop_multi_process_pool(self):
        self.embedder.stop_self_pool()

    @abstractmethod
    def __call__(
        self,
        corpus: Dict[str, Dict[str, Any]],
        queries: Dict[str, str],
        corpus_embd_save_dir: Optional[str] = None,
        ignore_identical_ids: bool = False,
        **kwargs,
    ) -> Dict[str, Dict[str, float]]:
        pass

```

FlagEmbedding 

EvalDenseRetriever ()

```

class EvalDenseRetriever(EvalRetriever):
    def __call__(
        self,
        corpus: Dict[str, Dict[str, Any]],
        queries: Dict[str, str],
        corpus_embd_save_dir: Optional[str] = None,
        ignore_identical_ids: bool = False,
        **kwargs,
    ):
        # 1. 
        corpus_ids = []
        corpus_texts = []
        for docid, doc in corpus.items():
            corpus_ids.append(docid)
            corpus_texts.append(doc["text"] if "title" not in doc
                                else f"{doc['title']} {doc['text']}".strip())

        queries_ids = []
        queries_texts = []
        for qid, query in queries.items():
            queries_ids.append(qid)
            queries_texts.append(query)

        # 2. 
        if corpus_embd_save_dir is not None:
            if os.path.exists(os.path.join(corpus_embd_save_dir, "doc.npy")) and not self.overwrite:
                corpus_emb = np.load(os.path.join(corpus_embd_save_dir, "doc.npy"))
            else:
                corpus_emb = self.embedder.encode_corpus(corpus_texts, **kwargs)
        else:
            corpus_emb = self.embedder.encode_corpus(corpus_texts, **kwargs)

        queries_emb = self.embedder.encode_queries(queries_texts, **kwargs)

        #  M3 dict 
        if isinstance(corpus_emb, dict):
            corpus_emb = corpus_emb["dense_vecs"]
        if isinstance(queries_emb, dict):
            queries_emb = queries_emb["dense_vecs"]

```

```

# 保存
if corpus_embd_save_dir is not None and ...:
    os.makedirs(corpus_embd_save_dir, exist_ok=True)
    np.save(os.path.join(corpus_embd_save_dir, "doc.npy"), corpus_emb)

# 3. 使用 FAISS 索引
faiss_index = index(corpus_embeddings=corpus_emb)
all_scores, all_indices = search(query_embeddings=queries_emb,
                                  faiss_index=faiss_index,
                                  k=self.search_top_k)

# 4. 返回结果
results = {}
for idx, (scores, indices) in enumerate(zip(all_scores, all_indices)):
    results[queries_ids[idx]] = {}
    for score, indice in zip(scores, indices):
        if indice != -1:
            if ignore_identical_ids and corpus_ids[indice] == queries_ids[idx]:
                continue
            results[queries_ids[idx]][corpus_ids[indice]] = float(score)

return results

```

EvalReranker

```

class EvalReranker:
    def __init__(self, reranker: AbsReranker, rerank_top_k: int = 100):
        self.reranker = reranker
        self.rerank_top_k = rerank_top_k

    def __call__(
        self,
        corpus: Dict[str, Dict[str, Any]],
        queries: Dict[str, str],
        search_results: Dict[str, Dict[str, float]],
        ignore_identical_ids: bool = False,
        **kwargs,
    ):
        # 1. □□□ rerank_top_k
        for qid in search_results:
            search_results[qid] = dict(
                sorted(search_results[qid].items(), key=lambda x: x[1], reverse=True)[:self.rerank_top_k]
            )

        # 2. □□□□□
        sentence_pairs = []
        pairs = []
        for qid in search_results:
            for docid in search_results[qid]:
                if ignore_identical_ids and qid == docid:
                    continue
                doc = corpus[docid]
                doc_text = doc["text"] if "title" not in doc else f"{doc['title']} {doc['text']}"
                sentence_pairs.append({"qid": qid, "docid": docid, "query": queries[qid], "doc": doc_text})
                pairs.append((queries[qid], doc_text))

        # 3. □□□□
        scores = self.reranker.compute_score(pairs)
        for i, score in enumerate(scores):
            sentence_pairs[i]["score"] = float(score)

        # 4. □□□□
        reranked_results = {qid: {} for qid in search_results}
        for pair in sentence_pairs:
            reranked_results[pair["qid"]][pair["docid"]] = pair["score"]

```

```
return reranked_results
```

4.5 유틸리티 (utils)

□□□□: [abc/evaluation/utils.py](#)

FAISS

□□□□

1. **evaluate_metrics**: `NDCG` `MAP` `Recall` `Precision`
2. `pytrec_eval`
3. `cut-off`
4. **evaluate_mrr**: Mean Reciprocal Rank
5. **evaluate_recall_cap**: Capped Recall


```

def index(
    index_factory: str = "Flat",
    corpus_embeddings: Optional[np.ndarray] = None,
    load_path: Optional[str] = None,
    device: Optional[str] = None
):
    """ⅡⅡ FAISS ⅡⅡ"""
    # ⅡⅡⅡⅡ
    faiss_index = faiss.index_factory(corpus_embeddings.shape[-1], index_factory, faiss.METRIC_IN

    # ⅡⅡ GPU ⅡⅡ
    if device is None and torch.cuda.is_available():
        try:
            co = faiss.GpuMultipleClonerOptions()
            co.shard = True
            co.useFloat16 = True
            faiss_index = faiss.index_cpu_to_all_gpus(faiss_index, co)
        except:
            print('faiss do not support GPU, please uninstall faiss-cpu, faiss-gpu and install fa

    # ⅡⅡⅡⅡ
    faiss_index.train(corpus_embeddings)
    faiss_index.add(corpus_embeddings)

    return faiss_index

def search(
    faiss_index: faiss.Index,
    k: int = 100,
    query_embeddings: Optional[np.ndarray] = None,
    load_path: Optional[str] = None
):
    """ⅡⅡ"""
    all_scores = []
    all_indices = []
    # ⅡⅡⅡⅡⅡⅡ OOMⅡ
    for i in tqdm(range(0, len(query_embeddings), 32), desc="Searching"):
        j = min(i + 32, len(query_embeddings))
        score, indice = faiss_index.search(query_embeddings[i:j].astype(np.float32), k=k)

```

```

all_scores.append(score)
all_indices.append(indice)
return np.concatenate(all_scores, axis=0), np.concatenate(all_indices, axis=0)

```

4.6 [] [] [] []

[] [] []: [abc/evaluation/arguments.py](#)

1. **AbsEvalArgs**: [] [] [] []

2. eval_name : [] [] [] []

3. dataset_dir : [] [] [] [] []

4. dataset_names : [] [] [] [] [] [] []

5. splits : [] [] [] []

6. output_dir : [] [] [] [] [] []

7. search_top_k / rerank_top_k

8. k_values : [] [] [] cut-off []

9. eval_metrics : [] [] [] [] [] [] []

10. []

11. **AbsEvalModelArgs**: [] [] [] []

12. embedder_name_or_path / embedder_model_class

13. reranker_name_or_path / reranker_model_class

14. [] [] [] [] [] batch size [] max length []

15. []

5. [] [] [] [] [] [] [] []

5.1 [] [] [] []

1. [] [] [] [] (Template Method)

[] [] [] []: [] [] [] [] [] [] []

AbsEmbedder AbsReranker AbsEmbedderModel 设计 - 设计 encode 设计 - 设计
 设计 encode_single_device 设计

设计:

```
# AbsEmbedder 设计
def encode(self, sentences, ...):
    # 设计 → 设计/设计 → 设计
    if instruction is not None:
        sentences = [self.get_detailed_instruct(...) for ...]

    if isinstance(sentences, str) or len(self.target_devices) == 1:
        return self.encode_single_device(...) # 设计
    else:
        return self.encode_multi_process(...)
```

2. 策略 (Strategy)

设计: 设计

AbsEmbedderModel 设计

```
if no_in_batch_neg_flag:
    compute_loss_func = self._compute_no_in_batch_neg_loss
else:
    if self.negatives_cross_device:
        compute_loss_func = self._compute_cross_device_neg_loss
    else:
        compute_loss_func = self._compute_in_batch_neg_loss
```

3. 工厂 (Factory)

设计: 设计 inference/auto_embedder.py 设计

设计 abc 设计 abc 设计

4. 构建器 (Builder)

设计: AbsEmbedderRunner 设计 AbsEvalRunner

Runner 设计 model 设计 dataset 设计 trainer 设计

5.2 设计

1. 关注点分离 (Separation of Concerns)

模块	功能
inference	推理
finetune	微调
evaluation	评估
abc	其他功能

2. 接口定义

- 接口: `AbsEmbedder / AbsReranker` 方法 `encode_single_device / compute_score_single_gpu`
- 接口: `AbsEmbedderModel` 方法
- 接口: `AbsEvalDataLoader` 方法

3. 实现

- 实现
- 接口 `**kwargs` 实现
- 实现

4. 配置

- 配置: GPU
- 配置: GPU 数量
- 配置:
- 配置 (`sub_batch_size`)
- 配置 (`share_memory`)
- 配置 (`corpus_embd_save_dir`)
- 配置: FP16/BF16 精度

5.

	In-batch negatives
	Cross-device negatives ()
	Knowledge Distillation
Matryoshka	MRL,
Batch	batch

5.3

接口:

AbsEmbedder (ABC)

- └ encode_queries()
- └ encode_corpus()
- └ encode()
- └ encode_single_device() [abstract]

AbsReranker (ABC)

- └ compute_score()
- └ compute_score_single_gpu() [abstract]

接口 (Embedder):

AbsEmbedderModel (ABC, nn.Module)

- └ encode() [abstract]
- └ compute_loss() [abstract]
- └ compute_score() [abstract]
- └ save() [abstract]
- └ forward()
- └ _compute_in_batch_neg_loss()
- └ _compute_cross_device_neg_loss()
- └ distill_loss()

AbsEmbedderTrainDataset (Dataset)

AbsEmbedderCollator (DataCollatorWithPadding)

AbsEmbedderTrainer (Trainer)

AbsEmbedderRunner (ABC)

接口 (Reranker):

[接口 Embedder接口]

接口:

AbsEvalRunner

- └ get_models()
- └ run()

AbsEvaluator

- └ __call__()
- └ evaluate_results()

AbsEvalDataLoader (ABC)

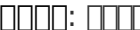
```
└─ load_corpus()
└─ load_queries()
└─ load_grels()
```

```
EvalRetriever (ABC)
└─ EvalDenseRetriever
```

```
EvalReranker
```



FlagEmbedding 

1. : 
2. : 
3. :  GPU 
4. : MRL

 FlagEmbedding 

03 inference module

FlagEmbedding



Mermaid Diagram



Mermaid Diagram

目录

名称	类型	模型	策略
FlagAutoModel	模型	Embedder	模型
FlagAutoReranker	模型	Reranker	模型
BaseEmbedder	Encoder-only	BERT	CLS/Mean Pooling
M3Embedder	Encoder-only	BGE-M3	模型
BaseLLMEmbedder	Decoder-only	LLM	Last Token Pooling
ICLLMEmbedder	Decoder-only	Few-shot	模型
BaseReranker	Encoder-only	Cross-Encoder	模型
LayerWiseLLMReranker	Decoder-only	LLM	模型

FlagEmbedding 目录 inference Embedder Reranker

- 1. 模型
- 2. 模型
- 3. Embedder 模型
- 4. Reranker 模型

1. 模型

/workspace/FlagEmbedding/inference/ 模型

```

inference/
├─ auto_embedder.py          # Embedder 
├─ auto_reranker.py          # Reranker 
├─ embedder/                  # Embedder 
│   ├─ encoder_only/          # -only 
│   │   ├─ base.py            # 
│   │   └─ m3.py              # BGE-M3 
│   └─ decoder_only/          # -only 
│       ├─ base.py            # 
│       ├─ icl.py             # ICL 
│       └─ pseudo_moe.py      #  MoE 
├─ model_mapping.py          # Embedder 
└─ __init__.py

├─ reranker/                  # Reranker 
│   ├─ encoder_only/          # -only
│   └─ decoder_only/          # -only
│       ├─ base.py            # 
│       └─ layerwise.py       # -wise 
│           └─ lightweight.py # 
└─ model_mapping.py          # Reranker 
└─ __init__.py

```

1.1

 abc.inference  - AbsEmbedder - Embedder  - AbsReranker - Reranker 



2.

FlagEmbedding 

2.1 FlagAutoModel - Embedder

 /workspace/FlagEmbedding/inference/auto_embedder.py

FlagAutoModel  Embedder  from_finetuned 

```
class FlagAutoModel:
    @classmethod
    def from_finetuned(
        cls,
        model_name_or_path: str,
        model_class: Optional[Union[str, EmbedderModelClass]] = None,
        normalize_embeddings: bool = True,
        use_fp16: bool = True,
        use_bf16: bool = False,
        query_instruction_for_retrieval: Optional[str] = None,
        devices: Optional[Union[str, List[str]]] = None,
        pooling_method: Optional[str] = None,
        trust_remote_code: Optional[bool] = None,
        query_instruction_format: Optional[str] = None,
        truncate_dim: Optional[int] = None,
        **kwargs,
    ):
        # 00000000...
        # 00000000...
        # 00000000 Embedder
```

□ □ □ □ □

1. `checkpoint`
2. `model_mapping`
3. `pooling_method, trust_remote_code, query_instruction_format`
4. `Embedder`

□ □ □ □ □

```
from FlagEmbedding import FlagAutoModel

# Load BGE-M3 model
model = FlagAutoModel.from_finetuned("BAAI/bge-m3")
```

2.2 FlagAutoReranker - Reranker

```
llm /workspace/FlagEmbedding/inference/auto reranker.py
```

```
FlagAutoReranker  FlagAutoModel  Reranker
```

```
class FlagAutoReranker:

    @classmethod
    def from_finetuned(
        cls,
        model_name_or_path: str,
        model_class: Optional[Union[str, RerankerModelClass]] = None,
        use_fp16: bool = False,
        trust_remote_code: Optional[bool] = None,
        **kwargs,
    ):
        # 0000000000...
```

2.3 Embedder

```
##### /workspace/FlagEmbedding/inference/embedder/model_mapping.py
```

2.3.1 EmbedderModelClass

```
class EmbedderModelClass(Enum):
    ENCODER_ONLY_BASE = "encoder-only-base"
    ENCODER_ONLY_M3 = "encoder-only-m3"
    DECODER_ONLY_BASE = "decoder-only-base"
    DECODER_ONLY_ICL = "decoder-only-icl"
    DECODER_ONLY_PSEUDO_MOE = "decoder-only-pseudo moe"
```

2.3.2 EmbedderConfig

```
@dataclass
class EmbedderConfig:
    model_class: Type[AbsEmbedder]
    pooling_method: PoolingMethod
    trust_remote_code: bool = False
    query_instruction_format: str = "{}{}"
```

2.3.3 PoolingMethod

```
class PoolingMethod(Enum):
    LAST_TOKEN = "last_token"
    CLS = "cls"
    MEAN = "mean"
```

2.3.4

AUTO_EMBEDDER_MAPPING

BGE	bge-m3 , bge-large-en-v1.5 , bge-base-en-v1.5 , bge-small-en-v1.5 , bge-large-zh-v1.5
Qwen3-Embedding	Qwen3-Embedding-0.6B , Qwen3-Embedding-4B , Qwen3-Embedding-8B
E5	e5-mistral-7b-instruct , e5-large-v2 , multilingual-e5-large
GTE	gte-Qwen2-7B-instruct , gte-large-en-v1.5 , gte-large-zh
SFR	SFR-Embedding-2_R , SFR-Embedding-Mistral

2.4 Reranker

/workspace/FlagEmbedding/inference/reranker/model_mapping.py

2.4.1 RerankerModelClass

```
class RerankerModelClass(Enum):
    ENCODER_ONLY_BASE = "encoder-only-base"
    DECODER_ONLY_BASE = "decoder-only-base"
    DECODER_ONLY_LAYERWISE = "decoder-only-layerwise"
    DECODER_ONLY_LIGHTWEIGHT = "decoder-only-lightweight"
```

2.4.2 RerankerConfig

```
@dataclass
class RerankerConfig:
    model_class: Type[AbsReranker]
    trust_remote_code: bool = False
```

2.4.3 Reranker

bge-reranker-base	-only
bge-reranker-large	-only
bge-reranker-v2-m3	-only
bge-reranker-v2-gemma	-only
bge-reranker-v2-minicpm-layerwise	-only layerwise
bge-reranker-v2.5-gemma2-lightweight	-only lightweight

3. Embedder

3.1 -only - BaseEmbedder

/workspace/FlagEmbedding/inference/embedder/encoder_only/base.py

BaseEmbedder -only Embedder AbsEmbedder

3.1.1

```

class BaseEmbedder(AbsEmbedder):
    DEFAULT_POOLING_METHOD = "cls"

    def __init__(
        self,
        model_name_or_path: str,
        normalize_embeddings: bool = True,
        use_fp16: bool = True,
        use_bf16: bool = False,
        query_instruction_for_retrieval: Optional[str] = None,
        query_instruction_format: str = "{}{}",
        devices: Optional[Union[str, List[str]]] = None,
        pooling_method: str = "cls",
        trust_remote_code: bool = False,
        cache_dir: Optional[str] = None,
        batch_size: int = 256,
        query_max_length: int = 512,
        passage_max_length: int = 512,
        convert_to_numpy: bool = True,
        truncate_dim: Optional[int] = None,
        **kwargs: Any,
    ):
        super().__init__(...)
        self.pooling_method = pooling_method

        #  tokenizer  model
        self.tokenizer = AutoTokenizer.from_pretrained(
            model_name_or_path,
            trust_remote_code=trust_remote_code,
            cache_dir=cache_dir
        )
        self.model = AutoModel.from_pretrained(
            model_name_or_path,
            trust_remote_code=trust_remote_code,
            cache_dir=cache_dir,
            dtype=self.get_model_torch_dtype(),
        )

```

FlagEmbedding [] [] [] [] [] []

□□□□□ - □□□□□□□□ FP32, FP16, BF16 - □□ pooling □□□□ CLS - □□□□ tokenizer □□

3.1.2 测试 - encode_single_device

```

@torch.no_grad()
def encode_single_device(
    self,
    sentences: Union[List[str], str],
    batch_size: int = 256,
    max_length: int = 512,
    convert_to_numpy: bool = True,
    device: Optional[str] = None,
    **kwargs: Any
):
    # 1. □□□□
    if device is None:
        device = self.target_devices[0]
    if device == "cpu":
        self.model.float()
    self.model.to(device)
    self.model.eval()

    # 2. □□□□□□□□□□□□□□□□
    all_inputs = []
    for start_index in trange(0, len(sentences), batch_size, desc='pre tokenize'):
        sentences_batch = sentences[start_index:start_index + batch_size]
        inputs_batch = self.tokenizer(
            sentences_batch,
            truncation=True,
            max_length=max_length,
            **kwargs
        )
        # □□□□□□□□□□
        inputs_batch = [{
            k: inputs_batch[k][i] for k in inputs_batch.keys()
        } for i in range(len(sentences_batch))]
        all_inputs.extend(inputs_batch)

    # 3. □□□□□□□□□□ padding□
    length_sorted_idx = np.argsort([-len(x['input_ids']) for x in all_inputs])
    all_inputs_sorted = [all_inputs[i] for i in length_sorted_idx]

    # 4. □□□□□ batch □□□□□ OOM□
    flag = False

```

```

while flag is False:
    try:
        inputs_batch = self.tokenizer.pad(
            all_inputs_sorted[: batch_size],
            padding=True,
            return_tensors='pt',
            **kwargs
        ).to(device)
        last_hidden_state = self.model(**inputs_batch, return_dict=True).last_hidden_state
        embeddings = self.pooling(last_hidden_state, inputs_batch['attention_mask'])
        flag = True
    except RuntimeError as e:
        batch_size = batch_size * 3 // 4 # □□□□□□□□ 75%
    except torch.cuda.OutOfMemoryError as e:
        batch_size = batch_size * 3 // 4

# 5. □□□□
all_embeddings = []
for start_index in tqdm(range(0, len(sentences), batch_size), desc="Inference Embeddings"):
    inputs_batch = all_inputs_sorted[start_index:start_index + batch_size]
    inputs_batch = self.tokenizer.pad(
        inputs_batch,
        padding=True,
        return_tensors='pt',
        **kwargs
    ).to(device)

    last_hidden_state = self.model(**inputs_batch, return_dict=True).last_hidden_state
    embeddings = self.pooling(last_hidden_state, inputs_batch['attention_mask'])
    embeddings = self._truncate_embeddings(embeddings)

    if self.normalize_embeddings:
        embeddings = torch.nn.functional.normalize(embeddings, dim=-1)

    embeddings = cast(torch.Tensor, embeddings)
    if convert_to_numpy:
        embeddings = self._convert_to_numpy(embeddings, device=device)
    all_embeddings.append(embeddings)

# 6. □□□□□□□□□□
if convert_to_numpy:

```

□ □ □ □ □ □

- ### 3.1.3 Pooling ☐

pooling - cls -

□ 76 □

3.2 BGE-M3 - M3Embedder

 /workspace/FlagEmbedding/inference/embedder/encoder_only/m3.py

M3Embedder  BGE-M3  embedding 

3.2.1

BGE-M3  - **Dense Embedding** -  - **Sparse Embedding** -  - **CoBERT** -  token  embedding 

3.2.2 encode_single_device

```

@torch.no_grad()
def encode_single_device(
    self,
    sentences: Union[List[str], str],
    batch_size: int = 256,
    max_length: int = 512,
    return_dense: bool = True,
    return_sparse: bool = False,
    return_colbert_vecs: bool = False,
    device: Optional[str] = None,
    **kwargs: Any
):
    # ... BaseEmbedder ...

    # []
    all_dense_embeddings, all_lexical_weights, all_colbert_vecs = [], [], []
    for start_index in tqdm(range(0, len(sentences), batch_size), desc="Inference Embeddings"):
        inputs_batch = all_inputs_sorted[start_index:start_index + batch_size]
        inputs_batch = self.tokenizer.pad(
            inputs_batch,
            padding=True,
            return_tensors='pt',
            **kwargs
        ).to(device)

        outputs = self.model(
            inputs_batch,
            return_dense=return_dense,
            return_sparse=return_sparse,
            return_colbert_vecs=return_colbert_vecs,
            truncate_dim=self.truncate_dim
        )

        # [] dense embedding
        if return_dense:
            all_dense_embeddings.append(
                self._convert_to_numpy(outputs['dense_vecs'], device=device)
            )

        # [] sparse embedding[]lexical weights[]

```

□ □ □ □ □

- 80 □

3.2.3 - compute_score

BGE-M3 

```
def compute_score(
    self,
    sentence_pairs: Union[List[Tuple[str, str]], Tuple[str, str]],
    batch_size: Optional[int] = None,
    max_query_length: Optional[int] = None,
    max_passage_length: Optional[int] = None,
    weights_for_different_modes: Optional[List[float]] = None,
    **kwargs: Any
) -> Dict[
    Literal["colbert", "sparse", "dense", "sparse+dense", "colbert+sparse+dense"],
    List[float]
]:
    # 
    if len(self.target_devices) == 1:
        return self.compute_score_single_device(...)
    else:
        return self.compute_score_multi_process(...)
```

compute_score_single_device

```

@torch.no_grad()
def compute_score_single_device(
    self,
    sentence_pairs: Union[List[Tuple[str, str]], Tuple[str, str]],
    batch_size: int = 256,
    max_query_length: int = 512,
    max_passage_length: int = 512,
    weights_for_different_modes: Optional[List[float]] = None,
    device: Optional[str] = None,
    **kwargs: Any
):
    # 1. 
    queries_inputs = _tokenize([pair[0] for pair in sentences_batch], max_query_length)
    passages_inputs = _tokenize([pair[1] for pair in sentences_batch], max_passage_length)

    # 2. 
    queries_output = self.model(
        queries_inputs,
        return_dense=True, return_sparse=True, return_colbert_vecs=True,
        return_sparse_embedding=True
    )
    passages_output = self.model(
        passages_inputs,
        return_dense=True, return_sparse=True, return_colbert_vecs=True,
        return_sparse_embedding=True
    )

    # 3. 
    dense_scores = self.model.compute_dense_score(q_dense, p_dense)
    sparse_scores = self.model.compute_sparse_score(q_sparse, p_sparse)
    colbert_scores = self.model.compute_colbert_score(q_colbert, p_colbert, q_mask)

    # 4. 
    if weights_for_different_modes is None:
        weights_for_different_modes = [1., 1., 1.] # [dense, sparse, colbert]
        weight_sum = 3

    # 5. 
    return {
        'colbert': colbert_scores,

```

```

'sparse': sparse_scores,
'dense': dense_scores,
'sparse+dense': (sparse * w1 + dense * w0) / (w0 + w1),
'colbert+sparse+dense': (colbert * w2 + sparse * w1 + dense * w0) / sum(weights)
}

```

3.3 □□□-only □□□□ - BaseLLMEmbedder

□□□□□ /workspace/FlagEmbedding/inference/embedder/decoder_only/base.py

BaseLLMEmbedder □□□□-only □□□□□□□□□□ last token pooling

3.3.1 Last Token Pooling

```

def last_token_pool(last_hidden_states: torch.Tensor,
                    attention_mask: torch.Tensor) -> torch.Tensor:
    # □□□□□□□ padding
    left_padding = (attention_mask[:, -1].sum() == attention_mask.shape[0])
    if left_padding:
        return last_hidden_states[:, -1] # □ padding□□□□□□□□ token
    else:
        # □ padding□□□□□□□□□□ token
        sequence_lengths = attention_mask.sum(dim=1) - 1
        batch_size = last_hidden_states.shape[0]
        return last_hidden_states[
            torch.arange(batch_size, device=last_hidden_states.device),
            sequence_lengths
        ]

```

□□□□□□ - □□□□□□/□ padding □□ - □□□□□□□□ token □□□□□□

3.4 ICL □□ - ICLLLMEmbedder

□□□□□□ /workspace/FlagEmbedding/inference/embedder/decoder_only/icl.py

ICLLLMEmbedder □□ Few-Shot □□□□□□□□□□□□□□□□

3.4.1 [] [] [] []

```

def set_examples(self, examples_for_task: Optional[List[dict]] = None):
    """[ ] [ ] Few-Shot [ ] [ ] [ ] [ ]"""
    if examples_for_task is None and self.examples_for_task is None:
        self.prefix = ''
    else:
        # [ ] [ ] [ ] [ ] [ ] [ ]
        eg_patis = []
        for example in examples_for_task or self.examples_for_task:
            eg_patis.append(
                self.get_detailed_example(
                    self.examples_instruction_format,
                    example.get('instruct', self.query_instruction_for_retrieval),
                    example.get('query', ''),
                    example.get('response', '')
                )
            )
        self.prefix = '\n\n'.join(eg_patis) + '\n\n'

```

3.4.2

```
@torch.no_grad()
def encode_queries_single_device(
    self,
    queries: Union[List[str], str],
    batch_size: int = 256,
    max_length: int = 512,
    convert_to_numpy: bool = True,
    device: Optional[str] = None,
    **kwargs: Any
):
    # 1. 
    if self.query_instruction_for_retrieval is not None:
        input_texts = [
            self.get_detailed_instruct(
                self.query_instruction_format,
                self.query_instruction_for_retrieval,
                query
            ) for query in queries
        ]

    # 2.  +  + 
    prefix_ids = self.tokenizer(self.prefix, add_special_tokens=False)['input_ids']
    suffix_ids = self.tokenizer(self.suffix, add_special_tokens=False)['input_ids']

    # 3. 
    new_max_length = (len(prefix_ids) + len(suffix_ids) + max_length + 8) // 8 * 8 + 8

    # 4.  BaseLLMEmbedder...
```

  +  + 

3.5 MoE - PseudoMoELLMEmbedder

 /workspace/FlagEmbedding/inference/embedder/decoder_only/pseudo_moe.py

PseudoMoELLMEmbedder  MoE  Mixture of Experts 

3.5.1 □□□□

```

def _resolve_domain(self, kwargs: Any) -> Optional[str]:
    """□ kwargs □□□□□□□□"""
    domain = kwargs.pop("domain_for_pseudo_moe", None)
    if domain is None:
        domain = kwargs.pop("domain", None)
    if domain is None:
        domain = self.domain_for_pseudo_moe
    return domain

@torch.no_grad()
def encode_single_device(
    self,
    sentences: Union[List[str], str],
    batch_size: int = 256,
    max_length: int = 512,
    convert_to_numpy: bool = True,
    device: Optional[str] = None,
    **kwargs: Any
):
    # □□□
    domain = self._resolve_domain(kwargs)

    # □□□□□□
    if domain is not None and hasattr(self.model, "set_domain"):
        self.model.set_domain(domain)

    # □□ forward □□□
    model_forward_kwargs = {"return_dict": True}
    if domain is not None:
        model_forward_kwargs["domain"] = domain

    # □□□□□ domain □□□
    last_hidden_state = self.model(**inputs_batch, **model_forward_kwargs).last_hidden_state
    # ...

```

4. Reranker

4.1 -only - BaseReranker

 /workspace/FlagEmbedding/inference/reranker/encoder_only/base.py

BaseReranker -only Reranker 

4.1.1 □□□□ - **compute_score_single_gpu**

```

@torch.no_grad()
def compute_score_single_gpu(
    self,
    sentence_pairs: Union[List[Tuple[str, str]], Tuple[str, str]],
    batch_size: Optional[int] = None,
    query_max_length: Optional[int] = None,
    max_length: Optional[int] = None,
    normalize: Optional[bool] = None,
    device: Optional[str] = None,
    **kwargs: Any
) -> List[float]:
    # 1. 
    all_inputs = []
    for start_index in trange(0, len(sentence_pairs), batch_size, desc="pre tokenize"):
        sentences_batch = sentence_pairs[start_index:start_index + batch_size]
        queries = [s[0] for s in sentences_batch]
        passages = [s[1] for s in sentences_batch]

        #  tokenize 
        queries_inputs_batch = self.tokenizer(
            queries,
            return_tensors=None,
            add_special_tokens=False,
            max_length=query_max_length,
            truncation=True,
            **kwargs
        )['input_ids']
        passages_inputs_batch = self.tokenizer(
            passages,
            return_tensors=None,
            add_special_tokens=False,
            max_length=max_length,
            truncation=True,
            **kwargs
        )['input_ids']

        #  <[BOS_never_used_51bce0c785ca2f68081bfa7d91973934]> + query + SEP + passage + SEP
        for q_inp, d_inp in zip(queries_inputs_batch, passages_inputs_batch):
            item = self.tokenizer.prepare_for_model(
                q_inp,

```

```

        d_inp,
        truncation='only_second', # [ ] [ ] [ ] [ ]
        max_length=max_length,
        padding=False,
    )
    all_inputs.append(item)

# 2. [ ] [ ] [ ] [ ]
length_sorted_idx = np.argsort([-len(x['input_ids']) for x in all_inputs])
all_inputs_sorted = [all_inputs[i] for i in length_sorted_idx]

# 3. [ ] [ ] [ ] batch [ ]
flag = False
while flag is False:
    try:
        test_inputs_batch = self.tokenizer.pad(all_inputs_sorted[:min(...)]).to(device)
        scores = self.model(**test_inputs_batch, return_dict=True).logits.view(-1, ).float()
        flag = True
    except (RuntimeError, torch.cuda.OutOfMemoryError):
        batch_size = batch_size * 3 // 4

# 4. [ ] [ ] [ ] [ ]
all_scores = []
for start_index in tqdm(range(0, len(all_inputs_sorted), batch_size), desc="Compute Scores"):
    inputs = self.tokenizer.pad(all_inputs_sorted[start_index:start_index + batch_size]).to(device)
    scores = self.model(**inputs, return_dict=True).logits.view(-1, ).float()
    all_scores.extend(scores.cpu().numpy().tolist())

# 5. [ ] [ ] [ ] [ ]
all_scores = [all_scores[idx] for idx in np.argsort(length_sorted_idx)]

# 6. Sigmoid [ ] [ ] [ ]
if normalize:
    all_scores = [sigmoid(score) for score in all_scores]

return all_scores

```

[] [] [] [] <[BOS_never_used_51bce0c785ca2f68081bfa7d91973934]> [] [] [SEP] [] [] [SEP]

4.2 [REDACTED]-only [REDACTED] - BaseLLMReranker

[REDACTED] /workspace/FlagEmbedding/inference/reranker/decoder_only/base.py

BaseLLMReranker [REDACTED] LLM [REDACTED] Reranker [REDACTED] "Yes" [REDACTED] "No" [REDACTED]

4.2.1 [REDACTED]

```
# [REDACTED]
if prompt is None:
    prompt = "Given a query A and a passage B, determine whether the passage contains an answer to the query A."

# [REDACTED]
item['input_ids'] = [bos_token] + query_inputs + sep + passage_inputs + sep + prompt_inputs
```

[REDACTED] [BOS] [REDACTED] [SEP] [REDACTED] [SEP] [REDACTED]

4.2.2 Last Logit Pooling

```
def last_logit_pool(logits: Tensor,
                    attention_mask: Tensor) -> Tensor:
    left_padding = (attention_mask[:, -1].sum() == attention_mask.shape[0])
    if left_padding:
        return logits[:, -1] # [REDACTED] padding [REDACTED] token
    else:
        sequence_lengths = attention_mask.sum(dim=1) - 1
        batch_size = logits.shape[0]
        return torch.stack([
            logits[i, sequence_lengths[i], :]
            for i in range(batch_size)
        ], dim=0)
```

[REDACTED] token [REDACTED] "Yes" [REDACTED] logit [REDACTED]

```
self.yes_loc = self.tokenizer('Yes', add_special_tokens=False)['input_ids'][0]
scores = last_logit_pool(logits, inputs['attention_mask'])
scores = scores[:, self.yes_loc] # [REDACTED] 'Yes' [REDACTED]
```

4.3 -wise - LayerWiseLLMReranker

 /workspace/FlagEmbedding/inference/reranker/decoder_only/layerwise.py

LayerWiseLLMReranker 

4.3.1

```
def compute_score_single_gpu(
    self,
    sentence_pairs: ...,
    cutoff_layers: Optional[List[int]] = None, # 
    ...
):
    #  cutoff_layers
    outputs = self.model(
        **batch_inputs,
        output_hidden_states=True,
        cutoff_layers=cutoff_layers # 
    )

    #  logits
    all_logits = outputs.logits # List[Tensor]
    tmp_all_scores = []
    for logits in all_logits:
        scores = last_logit_pool_layerwise(logits, inputs['attention_mask'])
        tmp_all_scores.append(scores.contiguous())

    # 
    all_scores = []
    for i in range(len(tmp_all_scores)):
        all_scores.append(tmp_all_scores[i].cpu().float().tolist())

    # 
    for i in range(len(all_scores)):
        all_scores[i] = [all_scores[i][idx] for idx in np.argsort(length_sorted_idx)]
        if normalize:
            all_scores[i] = [sigmoid(score) for score in all_scores[i]]

    return all_scores
```

[] [] [] [] [] [] [] [] [] [] [] []

4.4 [] [] [] [] - LightweightLLMReranker

[] [] [] [] /workspace/FlagEmbedding/inference/reranker/decoder_only/lightweight.py

LightweightLLMReranker [] [] [] Reranker [] [] [] [] [] [] [] [] [] [] [] []

4.4.1

```

def __init__(
    self,
    ...,
    compress_layers: List[int] = [8], # 
    compress_ratio: int = 1, #  1, 2, 4, 8
    ...
):
    #  CostWiseGemmaForCausalLM
    self.model = CostWiseGemmaForCausalLM.from_pretrained(...)

def compute_score_single_gpu(
    self,
    ...,
    compress_layers: Optional[List[int]] = None,
    compress_ratio: Optional[int] = None,
    ...
):
    # 
    outputs = self.model(
        **batch_inputs,
        output_hidden_states=True,
        compress_layer=compress_layers,
        compress_ratio=compress_ratio,
        query_lengths=query_lengths,
        prompt_lengths=prompt_lengths,
        cutoff_layers=cutoff_layers
    )

    # 
    scores = []
    for i in range(len(outputs.logits)):
        logits = last_logit_pool_lightweight(
            outputs.logits[i],
            outputs.attention_masks[i]
        )
        scores.append(logits.cpu().float().tolist())

```

5.

5.1

Padding		BaseEmbedder.encode_single_device
Batch Size	OOM	BaseEmbedder.encode_single_device
Last Token Pooling	Decoder-only	BaseLLMEmbedder
	BGE-M3	M3Embedder
	LLM Reranker	BaseLLMReranker

5.2

- [auto_embedder.py](#)
- [auto_reranker.py](#)
- [embedder/model_mapping.py](#)
- [embedder/encoder_only/base.py](#)
- [embedder/encoder_only/m3.py](#)
- [embedder/decoder_only/base.py](#)
- [reranker/encoder_only/base.py](#)
- [reranker/decoder_only/base.py](#)
- [reranker/decoder_only/layerwise.py](#)
- [reranker/decoder_only/lightweight.py](#)

04 finetune module

FlagEmbedding

Mermaid Diagram

Mermaid Diagram

		batch	AbsEmbedderModel
		GPU	AbsEmbedderModel
		→	distill_loss()
MRL			forward()
		→	EncoderOnlyEmbedderM3Model

-
-
- Embedder
- Reranker
-
-

- 

FlagEmbedding 

Embedder  **Reranker** 

```
finetune/
├─ embedder/
│   ├─ encoder_only/
│   │   ├─ base/      #  Encoder-Only 
│   │   └─ m3/        # BGE-M3 
│   └─ decoder_only/
│       ├─ base/      #  Decoder-Only 
│       └─ icl/        # In-Context Learning 
└─ reranker/
    ├─ encoder_only/
    │   └─ base/      #  Encoder-Only Reranker
    └─ decoder_only/
        ├─ base/      #  Decoder-Only Reranker
        └─ layerwise/ # 
```


AbsEmbedderModel

: </workspace/FlagEmbedding/abc/finetune/embedder/AbsModeling.py>

AbsEmbedderModel  Embedder  Embedder 



1. **encode(features)** - 
2. **compute_loss(scores, target)** - 
3. **compute_score(q_reps, p_reps)** -  Query-Passage 

4. forward(queries, passages, teacher_scores, no_in_batch_neg_flag) - []

--	--	--	--	--	--

```
# 1. [] queries [] passages
q_reps = self.encode(queries)
p_reps = self.encode(passages)

# 2. [] [] [] [] [] [] [] [] [] []
if no_in_batch_neg_flag:
    compute_loss_func = self._compute_no_in_batch_neg_loss
else:
    if self.negatives_cross_device:
        compute_loss_func = self._compute_cross_device_neg_loss
    else:
        compute_loss_func = self._compute_in_batch_neg_loss

# 3. [] [] [] []
scores, loss = compute_loss_func(q_reps, p_reps, teacher_targets=teacher_targets)
```

AbsRerankerModel

```
□□□□: /workspace/FlagEmbedding/abc/finetune/reranker/AbsModeling.py
```

```
AbsRerankerModel  Reranker
```

□□□□

- 
- 
- 

Embedder

1. Encoder-Only Base ☐

```
#####: /workspace/FlagEmbedding/finetune/embedder/encoder_only/base/
modeling.py
```

[] [] []: BiEncoderOnlyEmbedderModel

```
class BiEncoderOnlyEmbedderModel(AbsEmbedderModel):
    def __init__(
        self,
        base_model: PreTrainedModel,
        tokenizer: PreTrainedTokenizer = None,
        negatives_cross_device: bool = False,
        temperature: float = 1.0,
        sub_batch_size: int = -1,
        kd_loss_type: str = 'kl_div',
        use_mrl: bool = False,
        mrl_dims: List[int] = [],
        sentence_pooling_method: str = 'cls',
        normalize_embeddings: bool = False,
    )
```

[] [] [] []

1. `_sentence_embedding(last_hidden_state, attention_mask)`
2. [] [] [] [] [] []: `cls`, `mean`, `last_token`
3. `cls`: [] <[BOS_never_used_51bce0c785ca2f68081bfa7d91973934]> token []
4. `mean`: [] [] token [] []
5. `last_token`: [] [] [] [] [] [] [] token
6. `encode(features)`
7. [] [] [] [] [] [] [] OOM
8. [] [] MRL (Matryoshka Representation Learning)
9. [] [] embedding [] []
10. `compute_score(q_reps, p_reps)`
11. [] [] [] [] [] [] [] []
12. [] [] [] [] [] [] [] [] [] [] [] []

2. Encoder-Only M3 [] []

[] [] [] []: /workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py

```

000: EncoderOnlyEmbedderM3Model

```

BGE-M3  - **Dense Embedding:**  - **Sparse Embedding:** 
 - **ColBERT:**  token 

□ □ □ □

1. `□□□□□□`
2. `_dense_embedding()` : `□□□□□□□□□□`
3. `_sparse_embedding()` : `□□□□□□□□□□`
4. `_colbert_embedding()` : `□□ token □□□□`
5. `□□□□□□□ ``python # □□□□□□□ dense_scores, dense_loss = compute_loss_func(...)`
`sparse_scores, sparse_loss = compute_loss_func(...) colbert_scores, colbert_loss`
`= compute_loss_func(...)`

```
#  ensemble_scores, ensemble_loss = compute_loss_func(...)
```

```
# loss = (loss + ensemble_loss + 0.1 * sparse_loss + Colbert_loss) / 4 ``
```

1.  **(Self-Distillation)**
2. 
3. 

3. Decoder-Only Base ☐☐

```

#####: /workspace/FlagEmbedding/finetune/embedder/decoder_only/base/
modeling.py

```

```
BiDecoderOnlyEmbedderModel
```

-  Encoder-Only 
- 
-  last token 

4. Decoder-Only ICL ☐

```
#####: /workspace/FlagEmbedding/finetune/embedder/decoder_only/icl/modeling.py
```

```

000: BiDecoderOnlyEmbedderICLModel

```

- In-Context Learning
-

Reranker ☐☐☐☐

1. Encoder-Only Base ☐

```
#####: /workspace/FlagEmbedding/finetune/reranker/encoder_only/base/modeling.py
```

□□□: **CrossEncoderModel**

```
class CrossEncoderModel(AbsRerankerModel):
    def encode(self, features):
        return self.model(**features, return_dict=True).logits
```

-  Cross-Encoder 
- 
-  logits 

2. Decoder-Only Base

```
#####: /workspace/FlagEmbedding/finetune/reranker/decoder_only/base/modeling.py
```

□□□: **CrossDecoderModel**

```
def encode(self, features):
    outputs = self.model(input_ids=features['input_ids'],
                          attention_mask=features['attention_mask'],
                          position_ids=features['position_ids'] if 'position_ids' in features.keys() else None,
                          output_hidden_states=True)
    scores = outputs.logits[:, -1, self.yes_loc]
    return scores.contiguous()
```

-  Decoder  Reranker
-  "Yes" token       

3. Decoder-Only Layerwise

: /workspace/FlagEmbedding/finetune/reranker/decoder_only/layerwise/modeling.py

: `CrossDecoderModel` (Layerwise )

- 
- 
- 

```
def forward(self, pair, teacher_scores=None):
    ranker_logits = self.encode(pair)

    if self.training:
        loss = 0
        # 
        for logits in ranker_logits:
            loss += self.compute_loss(...)

        # : 
        if teacher_scores is None:
            teacher_scores = ranker_logits[-1]
            teacher_targets = torch.softmax(teacher_scores.detach(), dim=-1)
            for logits in ranker_logits[:-1]:
                loss += distillation_loss
```



1.



`AbsEmbedderModel` 

□□	□□	□□
□□□□	_compute_no_in_batch_neg_loss	□□□□□□□□□□
□□□□	_compute_in_batch_neg_loss	□□□□ batch □□□□□□□□
□□□□	_compute_cross_device_neg_loss	□□□□□□□□□□□□

□□□□

```
score(q, p) = q · p / temperature
loss = CrossEntropy(score, target)
```

2. □□□□ (In-Batch Negatives)

□□□□: [AbsEmbedderModel._compute_in_batch_neg_loss](#)

□□□□

1. □□ batch □□□□ query□batch □□□□ passage □□□□
2. □□□□ query □□□□□□□□ score □□
3. □□□□□□: target = idx * group_size

```
def _compute_in_batch_neg_loss(self, q_reps, p_reps, teacher_targets=None, ...):
    # □□□□ query-passage □□□□
    scores = self.compute_score(q_reps, p_reps) # (batch_size, batch_size * group_size)

    # □□□□□□ query □□□□□□□□
    idxs = torch.arange(q_reps.size(0), device=q_reps.device)
    targets = idxs * group_size # (batch_size)

    # □□□□
    loss = self.compute_loss(scores, targets)
    return scores, loss
```

3. □□□□□□ (Cross-Device Negatives)

□□□□: [AbsEmbedderModel._compute_cross_device_neg_loss](#)

□□□□

1. □□ torch.distributed □□□□□□□ embedding
2. □□□□□□ embedding □□□□□
3. □□□□□□□□□□□□

```
def _compute_cross_device_neg_loss(self, q_reps, p_reps, ...):
    # □□□□□□□□ embedding
    cross_q_reps = self._dist_gather_tensor(q_reps)
    cross_p_reps = self._dist_gather_tensor(p_reps)

    # □□□□□□□□□□□□
    cross_scores = self.compute_score(cross_q_reps, cross_p_reps)

    # □□□□□
    cross_idxes = torch.arange(cross_q_reps.size(0))
    cross_targets = cross_idxes * group_size
    loss = self.compute_loss(cross_scores, cross_targets)
    return cross_scores, loss
```

□□□□□□□□

```
def _dist_gather_tensor(self, t):
    all_tensors = [torch.empty_like(t) for _ in range(self.world_size)]
    dist.all_gather(all_tensors, t)
    all_tensors[self.process_rank] = t # □□□□□□□□□□□□□□□□
    all_tensors = torch.cat(all_tensors, dim=0)
    return all_tensors
```

4. □□□□ (KD) □□

□□□□: [AbsEmbedderModel.distill_loss](#)

□□ **KD** □□□□

1. **kl_div** : □□ KL □□ python loss = - mean(sum(log_softmax(student) * teacher_target))
2. **m3_kd_loss** : M3 □□□□□□□□ KD
3. □□□□□□□□□□

4. mask

```
def distill_loss(kd_loss_type, teacher_targets, student_scores, group_size=None):
    if kd_loss_type == 'kl_div':
        return - torch.mean(
            torch.sum(torch.log_softmax(student_scores, dim=-1) * teacher_targets, dim=-1)
        )
    elif kd_loss_type == 'm3_kd_loss':
        labels = torch.arange(student_scores.size(0)) * group_size
        loss = 0
        mask = torch.zeros_like(student_scores)
        for i in range(group_size):
            temp_target = labels + i
            temp_scores = student_scores + mask
            temp_loss = F.cross_entropy(temp_scores, temp_target, reduction="none")
            loss += torch.mean(teacher_targets[:, i] * temp_loss)
            mask = torch.scatter(mask, dim=-1, index=temp_target.unsqueeze(-1),
                                value=torch.finfo(student_scores.dtype).min)
        return loss
```

5. MRL (Matryoshka Representation Learning)

□□□□: AbsEmbedderModel.forward

□ □ □ □

- □ embedding □□□□□□□□□□□□
- □□□□□□□□□□□□
- □□□□□□□□□□

```
if self.use_mrl:
    # 000000000000
    all_loss = torch.tensor(0.0, device=device)
    for dim_q_reps, dim_p_reps in zip(q_reps, p_reps):
        _, mrl_loss = compute_loss_func(dim_q_reps, dim_p_reps, teacher_targets=teacher_targets)
        all_loss += mrl_loss
    loss = all_loss / len(self.mrl_dims)
```

□□□

□□□□

1. □□□□ (AbsDataset)
- ↓
2. □□□□ (Collator)
- ↓
3. □□□□ (encode)
- ↓
4. □□□□ (compute_score)
- ↓
5. □□□□ (compute_loss)
- ↓
6. □□□□ & □□□□

□□□□

□□□□: </workspace/FlagEmbedding/abc/finetune/embedder/AbsDataset.py>

□□□

1. **AbsEmbedderTrainDataset** : □□□□□
2. □ json/jsonl □□□□
3. □□□□□□□
4. □□□□□□□□
5. **AbsEmbedderSameDatasetTrainDataset** : □□□□□□□
6. □□ batch □□□□□□□□
7. □□□□ batch size
8. □□ no_in_batch_neg □□
9. **AbsEmbedderCollator** : □□□□□
10. tokenization
11. padding
12. □□□□□□

□□□□

```
{
  "query": "□□□□□□□□",
  "pos": ["□□□□□□□□□□□□□□..."],
  "neg": ["□□□□□□□□...", "□□□□□□□□..."],
  "pos_scores": [0.95], // □□□□□ KD
  "neg_scores": [0.1, 0.05] // □□□□□ KD
}
```

□□□□□

□□□□: /workspace/FlagEmbedding/finetune/embedder/encoder_only/base/trainer.py□□□□: **EncoderOnlyEmbedderTrainer**

□□□ AbsEmbedderTrainer □□□□□ _save □□□

```
class EncoderOnlyEmbedderTrainer(AbsEmbedderTrainer):
    def _save(self, output_dir: Optional[str] = None, state_dict=None):
        # □□□□
        self.model.save(output_dir)
        # □□ tokenizer
        self.tokenizer.save_pretrained(output_dir)
        # □□□□□□□
        torch.save(self.args, os.path.join(output_dir, "training_args.bin"))
```

EmbedderOutput

EmbedderOutput

```
@dataclass
class EmbedderOutput(ModelOutput):
    q_reps: Optional[Tensor] = None      # Query  []
    p_reps: Optional[Tensor] = None      # Passage  []
    loss: Optional[Tensor] = None        #  []
    scores: Optional[Tensor] = None      #  []
```

RerankerOutput

```
@dataclass
class RerankerOutput(ModelOutput):
    loss: Optional[Tensor] = None        #  []
    scores: Optional[Tensor] = None      #  []
```

FlagEmbedding

FlagEmbedding  1. :  2. :  3. : MRL  4. : Encoder-only  Decoder-only  5. : BGE-M3 

05 evaluation module

FlagEmbedding

FlagEmbedding

Mermaid Diagram

数据集

Mermaid Diagram

数据集

数据集	数据集	数据集	数据集	数据集
MTEB	数据集	数据集	数据集	数据集
BEIR	数据集	数据集	15数据集	数据集
MSMARCO	数据集	数据集	数据集	passage/document
MIRACL	数据集	18数据集	数据集	数据集
MKQA	数据集	26数据集	数据集	数据集
MLDR	数据集	13数据集	数据集	数据集
AIR-Bench	数据集	数据集	数据集	数据集
BRIGHT	数据集	数据集	数据集	short/long数据集

数据集

- 1. 数据集
- 2. 数据集
- 3. 数据集
- 3.1 MTEB / C-MTEB
- 3.2 BEIR
- 3.3 MSMARCO
- 3.4 MIRACL
- 3.5 MKQA
- 3.6 MLDR
- 3.7 AIR-Bench
- 3.8 BRIGHT
- 4. 数据集

1.

FlagEmbedding  + 

$$\square\square\square\square \rightarrow \square\square/\square\square\square \rightarrow \square\square\square\square\square\square$$

--	--	--	--	--	--	--

```

FlagEmbedding/
├─ abc/evaluation/      # 000000
└─ evaluation/
   ├─ air_bench/       # AIR Bench 00
   ├─ beir/            # BEIR 00
   ├─ bright/          # BRIGHT 00
   ├─ custom/          # 000000
   ├─ miracl/          # MIRACL 00
   ├─ mkqa/            # MKQA 00
   ├─ mlldr/           # MLDR 00
   ├─ msmarco/         # MSMARCO 00
   └─ mteb/            # MTEB 00

```

2. 六六六

2.1 AbsEvalRunner runner.py

[illegible]

```
class AbsEvalRunner:
    def __init__(self, eval_args, model_args):
        # 初始化
        self.retriever, self.reranker = self.load_retriever_and_reranker()
        self.data_loader = self.load_data_loader()
        self.evaluator = self.load_evaluator()
```

```
ite=False):
```

□□□□□□□□ 5. □□□□□□

[ker searcher.py](#)
$$\begin{array}{|c|c|c|c|c|c|} \hline & & & & & \\ \hline \end{array} - \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline & & & & & & & & \\ \hline \end{array}$$

y

```
cache_dir=None, ...):
```

```
load_qrels(): □□□□□□ -
```

arguments.py

3. □□□□□□

3.1 MTEB / C-MTEB

[evaluation/mteb/runner.py](#)

□□ - □□□□ mteb □ - □□□□□□□□□□□□□□ - □□□□ - □□□□□□□□

□□□□ MTEBEvalRunner

1. `mteb.get_tasks()` 2. 3. 4.

□ □ □ □ □ □ □

```
{
  "task_type": {
    "task_name": score,
    "Avg": average_score
  },
  "Avg": overall_average
}
```

3.2 BEIR

□□□□ - [evaluation/beir/runner.py](#) - [evaluation/beir/data_loader.py](#)

[illegible]

□□□□□□ - arguana, climate-fever, cqadupstack, dbpedia-entity - fever, fiqa, hotpotqa, msmarco, nfcorpus, nq - quora, scidocs, scifact, trec-covid, webis-touche2020

```

██████ - BEIREvalRunner : BEIR ████████ - BEIREvalDataLoader : BEIR ████████

```

prompts.py - cqadupstack

3.3 MSMARCO

□□□□ - [evaluation/msmarco/runner.py](#) - [evaluation/msmarco/data_loader.py](#)

□□□ - □□□□□□□□□□□□ - □□ passage □ document □□□□ - □□ dev, dl19, dl20 □□□□

```

MSMARCOEvalRunner : MSMARCO
MSMARCOEvalDataLoader : MSMARCO

```

[REDACTED] - passage [REDACTED] Tevatron/msmarco-passage-corpus - document [REDACTED] irds/msmarco-document

3.4 MIRACL

[REDACTED] - [evaluation/miracle/runner.py](#) - [evaluation/miracle/data_loader.py](#)

[REDACTED] - [REDACTED] - [REDACTED] 18 [REDACTED] - [REDACTED] train [REDACTED] dev [REDACTED]

[REDACTED] - ar, bn, en, es, fa, fi, fr, hi, id, ja, ko, ru, sw, te, th, zh, de, yo

[REDACTED] - MIRACLEvalRunner : MIRACL [REDACTED] - MIRACLEvalDataLoader : MIRACL [REDACTED]

[REDACTED] miracl/miracle-corpus (HuggingFace)

3.5 MKQA

[REDACTED] - [evaluation/mkqa/runner.py](#) - [evaluation/mkqa/data_loader.py](#)

[REDACTED] - [REDACTED] - [REDACTED] 26 [REDACTED]/[REDACTED] - [REDACTED]

[REDACTED] - en, ar, fi, ja, ko, ru, es, sv, he, th, da, de, fr, it, nl, pt, hu, vi, ms, km, no, tr, zh_cn, zh_hk, zh_tw

[REDACTED] - MKQAEvalRunner : MKQA [REDACTED] - MKQAEvalDataLoader : MKQA [REDACTED]

[REDACTED] - [REDACTED] BelR [REDACTED] nq [REDACTED] - [REDACTED] [utils/normalize_text.py](#)

[REDACTED] Shitao/bge-m3-data (HuggingFace)

3.6 MLDR

[REDACTED] - [evaluation/mldr/runner.py](#) - [evaluation/mldr/data_loader.py](#)

[REDACTED] - [REDACTED] - [REDACTED] 13 [REDACTED] - [REDACTED] train, dev, test [REDACTED]

[REDACTED] - ar, de, en, es, fr, hi, it, ja, ko, pt, ru, th, zh

[REDACTED] - MLDREvalRunner : MLDR [REDACTED] - MLDREvalDataLoader : MLDR [REDACTED]

[REDACTED] Shitao/MLDR (HuggingFace)

3.7 AIR-Bench

[REDACTED] [evaluation/air_bench/runner.py](#)

[REDACTED] - [REDACTED] air-bench [REDACTED] - [REDACTED] - [REDACTED]

[REDACTED] AIRBenchEvalRunner

[REDACTED] 1. [REDACTED] AIRBench [REDACTED] 2. [REDACTED] 3. [REDACTED]

3.8 BRIGHT

[REDACTED] [evaluation/bright/runner.py](#)

[REDACTED] - [REDACTED] short [REDACTED] long - [REDACTED] - [REDACTED]

[REDACTED] - BrightEvalRunner : BRIGHT [REDACTED] - BrightEvalDenseRetriever : BRIGHT [REDACTED] -
BrightShortEvalDataLoader / BrightLongEvalDataLoader : [REDACTED]

[REDACTED] - [REDACTED] [prompts.py](#) - [REDACTED] retriever_qrels [REDACTED]

4. [REDACTED]

4.1 [REDACTED]

1. [REDACTED]
2. [REDACTED]
3. [REDACTED]
4. [REDACTED]
5. [REDACTED]
6. [REDACTED] Faiss [REDACTED]
7. [REDACTED]
8. [REDACTED]
9. [REDACTED]
10. [REDACTED]
11. [REDACTED]
12. [REDACTED] qrels
13. [REDACTED]
14. [REDACTED]JSON [REDACTED] Markdown[REDACTED]

4.2



- **NDCG@k**: Normalized Discounted Cumulative Gain
- **MAP**: Mean Average Precision
- **Recall@k**: 
- **Precision@k**: 
- **MRR**: Mean Reciprocal Rank
- **Recall Cap**: 

 [utils.py](#) 

4.3

JSON 

```
{
  "model_name": {
    "reranker_name": {
      "split": {
        "ndcg_at_10": 0.5,
        "recall_at_10": 0.6,
        ...
      }
    }
  }
}
```

Markdown 

5.

FlagEmbedding 

1. 
2. 
3.  top-k 

4. □□□□□□□□□□□□□□□□□□
5. □□□□□□□□□□□□□□□□□□

06 key algorithms

FlagEmbedding ☐☐☐☐☐☐

--	--	--	--	--	--

Mermaid Diagram

CLS Pooling	Pooling	last_hidden_state[:, 0]			
Mean Pooling	Pooling				
Last Token	Pooling	token		Decoder	
		batch		batch	
		GPU			
KL		teacher_scores			
M3				M3	
Dense		pooling			
Sparse					
ColBERT		token			

BGE-M3

Mermaid Diagram

FlagEmbedding ,MRLColBERT

1.

1.1

FlagEmbedding Transformer BERTRoBERTa

```
/workspace/FlagEmbedding/finetune/embedder/encoder only/base/modeling.py:94-95 - /
```

```
<[BOS never used 51bce0c785ca2f68081bfa7d91973934]> token [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
```

```
96-101 - /workspace/FlagEmbedding/inference/embedder/encoder_only/base.py:303-306
```

1.2.3 Last Token Pooling

Last Token pooling token Decoder-Only

```

102-113

```

□□□□□

```
elif self.sentence_pooling_method == "last_token":
    left_padding = attention_mask[:, -1].sum() == attention_mask.shape[0]
    if left_padding:
        return last_hidden_state[:, -1]
    else:
        sequence_lengths = attention_mask.sum(dim=1) - 1
        batch_size = last_hidden_state.shape[0]
        return last_hidden_state[
            torch.arange(batch_size, device=last_hidden_state.device),
            sequence_lengths,
        ]
```

[illegible]

1.3 〇〇〇〇〇

```

000000 - /workspace/FlagEmbedding/finetune/embedder/encoder_only/base/modeling.py:
104-105 - /workspace/FlagEmbedding/inference/embedder/encoder_only/base.py:263-264

```

□ □ □ □ □

```
if self.normalize_embeddings:
    all_p_reps = torch.nn.functional.normalize(all_p_reps, dim=-1)
```

□□□□□□ - □□□□□□□□ - □□□□□□ - □□□□□□

FlagEmbedding MRL

```

/workspace/FlagEmbedding/inference/embedder/encoder_only/base.py:262

```

```
embeddings = self.truncate_embeddings(embeddings)
```



```

def _compute_cross_device_neg_loss(self, q_reps, p_reps, teacher_targets=None, compute_score_func=None,
    group_size = p_reps.size(0) // q_reps.size(0)

    cross_q_reps = self._dist_gather_tensor(q_reps)
    cross_p_reps = self._dist_gather_tensor(p_reps)

    if compute_score_func is None:
        cross_scores = self.compute_score(cross_q_reps, cross_p_reps)
    else:
        cross_scores = compute_score_func(cross_q_reps, cross_p_reps, **kwargs)

    # □□□□
    ...

    return cross_scores, loss

```

□□□□□□□□

```

def _dist_gather_tensor(self, t: Optional[torch.Tensor]):
    if t is None:
        return None
    t = t.contiguous()

    all_tensors = [torch.empty_like(t) for _ in range(self.world_size)]
    dist.all_gather(all_tensors, t)

    all_tensors[self.process_rank] = t
    all_tensors = torch.cat(all_tensors, dim=0)

    return all_tensors

```

□□□□ - □□□□□□□□□□□□ - □□□□□ = □□□ × batch_size - □□□□ PyTorch Distributed □□□□

3.3 □□□□□□□

□□□□ no_in_batch_neg_flag=True □□□□□□□□□□□□□□

□□□□□□ - /workspace/FlagEmbedding/abc/finetune/embedder/AbsModeling.py:149-169

□□□□□□

```
def _compute_no_in_batch_neg_loss(self, q_reps, p_reps, teacher_targets=None, compute_score_func=None):
    group_size = p_reps.size(0) // q_reps.size(0)

    local_scores = self.compute_local_score(q_reps, p_reps, compute_score_func, **kwargs)

    if teacher_targets is not None:
        loss = self.distill_loss(self.kd_loss_type, teacher_targets, local_scores, group_size=group_size)
        if self.kd_loss_type == "kl_div":
            local_targets = torch.zeros(local_scores.size(0), device=local_scores.device, dtype=torch.float)
            loss += self.compute_loss(local_scores, local_targets)
    else:
        local_targets = torch.zeros(local_scores.size(0), device=local_scores.device, dtype=torch.float)
        loss = self.compute_loss(local_scores, local_targets)

    return local_scores, loss
```

4.

Knowledge Distillation, KD

4.1 KL □□□□

KL

```

000000 - /workspace/FlagEmbedding/abc/finetune/embedder/AbsModeling.py:319-324

```

□□□□□

```
if kd_loss_type == 'kl_div':
    return - torch.mean(
        torch.sum(torch.log_softmax(student_scores, dim=-1) * teacher_targets, dim=-1)
    )
```

1. teacher_scores 2. softmax teacher_targets 3. student scores 4. KL 5.

□ □ □ □ □ □ □ □

```
if teacher_scores is not None:
    teacher_scores = torch.tensor(teacher_scores, device=device)
    teacher_scores = teacher_scores.view(batch_size, -1).detach()
    teacher_targets = F.softmax(teacher_scores, dim=-1)
else:
    teacher_targets = None
```

4.2 M3 KD

□□ M3 □□□□□□□□□□□□□□□□

```

00000 - /workspace/FlagEmbedding/abc/finetune/embedder/AbsModeling.py:325-340

```

□□□□□

```
elif kd_loss_type == 'm3_kd_loss':
    labels = torch.arange(student_scores.size(0), device=student_scores.device, dtype=torch.long)
    labels = labels * group_size

    loss = 0
    mask = torch.zeros_like(student_scores)
    for i in range(group_size):
        temp_target = labels + i
        temp_scores = student_scores + mask
        temp_loss = F.cross_entropy(temp_scores, temp_target, reduction="none")
        loss += torch.mean(teacher_targets[:, i] * temp_loss)
        mask = torch.scatter(mask, dim=-1, index=temp_target.unsqueeze(-1),
                             value=torch.finfo(student_scores.dtype).min)

    return loss
```

5. MRL Matryoshka Representation Learning

11

[illegible]

5.1

MRL  k 

5.2

 - /workspace/FlagEmbedding/finetune/embedder/encoder_only/base/modeling.py:

91-102 - /workspace/FlagEmbedding/abc/finetune/embedder/AbsModeling.py:264-295

 **MRL** 

```
def encode(self, features):
    # ...  ...
    if self.use_mrl:
        p_reps_list = []
        ori_dim = all_p_reps.size(-1)
        for dim in self.mrl_dims:
            if dim > ori_dim:
                logger.warning(f"MRL dim {dim} is larger than original dimension {ori_dim}, using
                dim = min(dim, ori_dim)
                dim_p_reps = all_p_reps[:, :dim]
                if self.normalize_embeddings:
                    dim_p_reps = torch.nn.functional.normalize(dim_p_reps, dim=-1)
                p_reps_list.append(dim_p_reps.contiguous())
        return p_reps_list
    else:
        if self.normalize_embeddings:
            all_p_reps = torch.nn.functional.normalize(all_p_reps, dim=-1)
        return all_p_reps.contiguous()
```

 **MRL** 

```
if self.use_mrl:
    all_loss = torch.tensor(0.0, device=device)
    for dim_q_reps, dim_p_reps in zip(q_reps, p_reps):
        _, mrl_loss = compute_loss_func(dim_q_reps, dim_p_reps, teacher_targets=teacher_targets)
        all_loss += mrl_loss
    loss = all_loss / len(self.mrl_dims)
else:
    scores, loss = compute_loss_func(q_reps, p_reps, teacher_targets=teacher_targets)
```

MRL  -  -  -  - 

6. CoBERT M3

BGE-M3  FlagEmbedding  CoBERT 

6.1

M3  encoder  - `sparse_linear` :  - `colbert_linear` : 
CoBERT 

 - `/workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py:70-72`



```
if not self.unified_finetuning:
    self.model = base_model['model']
    self.colbert_linear = None
    self.sparse_linear = None
else:
    self.model = base_model['model']
    self.colbert_linear = base_model['colbert_linear']
    self.sparse_linear = base_model['sparse_linear']
```

6.2 Dense

 pooling 

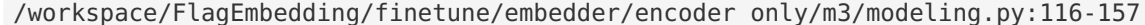
 - `/workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py:81-113`

```
def _dense_embedding(self, last_hidden_state, attention_mask):
    if self.sentence_pooling_method == "cls":
        return last_hidden_state[:, 0]
    elif self.sentence_pooling_method == "mean":
        s = torch.sum(
            last_hidden_state * attention_mask.unsqueeze(-1).float(), dim=1
        )
        d = attention_mask.sum(dim=1, keepdim=True).float()
        return s / d
    # ...
```

6.3 Sparse

 token 

 -

 /workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py:116-157



```

def _sparse_embedding(self, hidden_state, input_ids, return_embedding: bool = True):
    token_weights = torch.relu(self.sparse_linear(hidden_state))
    if not return_embedding:
        return token_weights

    if self.training:
        sparse_embedding = torch.zeros(
            input_ids.size(0), input_ids.size(1), self.vocab_size,
            dtype=token_weights.dtype,
            device=token_weights.device
        )
        sparse_embedding = torch.scatter(sparse_embedding, dim=-1, index=input_ids.unsqueeze(-1),
                                         src=token_weights)
        sparse_embedding = torch.max(sparse_embedding, dim=1).values
    else:
        sparse_embedding = torch.zeros(
            input_ids.size(0), self.vocab_size,
            dtype=token_weights.dtype,
            device=token_weights.device
        )
        sparse_embedding = sparse_embedding.scatter_reduce(
            dim=-1, index=input_ids, src=token_weights.squeeze(-1), reduce="amax"
        )

    unused_tokens = [
        self.tokenizer.cls_token_id, self.tokenizer.eos_token_id,
        self.tokenizer.pad_token_id, self.tokenizer.unk_token_id
    ]
    sparse_embedding[:, unused_tokens] *= 0.
    return sparse_embedding

```

[] [] [] [] [] - [] [] [] [] [] - [] [] [] [] - [] [] [] [] [] [] [] [] [] - [] [] BM25 [] [] [] [] [] []

[] [] [] [] [] [] [] [] [] [] [] token [] [] [] [] [] [] []

```
/workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py:150, 171
```

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----

6.5

M3 

 -

`/workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py:374-476`



```

def forward(self, queries, passages, teacher_scores=None, no_in_batch_neg_flag=False):
    q_dense_vecs, q_sparse_vecs, q_colbert_vecs = self.encode(queries)
    p_dense_vecs, p_sparse_vecs, p_colbert_vecs = self.encode(passages)

    if self.training:
        # [ ] [ ] [ ] [ ] [ ]
        dense_scores, loss = compute_loss_func(
            q_dense_vecs, p_dense_vecs, teacher_targets=teacher_targets,
            compute_score_func=self.compute_dense_score
        )

    if self.unified_finetuning:
        # [ ] [ ] [ ] [ ] [ ]
        sparse_scores, sparse_loss = compute_loss_func(
            q_sparse_vecs, p_sparse_vecs, teacher_targets=teacher_targets,
            compute_score_func=self.compute_sparse_score
        )

        # [ ] [ ] ColBERT [ ] [ ]
        colbert_scores, colbert_loss = compute_loss_func(
            q_colbert_vecs, p_colbert_vecs, teacher_targets=teacher_targets,
            compute_score_func=self.compute_colbert_score,
            q_mask=self._get_queries_attention_mask(queries)
        )

        # [ ] [ ] [ ] [ ]
        ensemble_scores, ensemble_loss = compute_loss_func(
            q_dense_vecs, p_dense_vecs, teacher_targets=teacher_targets,
            compute_score_func=self.ensemble_score,
            dense_scores=dense_scores,
            sparse_scores=sparse_scores,
            colbert_scores=colbert_scores
        )

        loss = (loss + ensemble_loss + 0.1 * sparse_loss + colbert_loss) / 4

        # [ ] [ ] [ ]
        if self.use_self_distill and self.step > self.self_distill_start_step:
            self_teacher_targets = torch.softmax(ensemble_scores.detach(), dim=-1)
            # ... [ ] [ ] [ ] [ ] [ ] ...

```

```
return EmbedderOutput(loss=loss)
```

6.6

M3 

 -

[/workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py:173-189](#) - [/workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/modeling.py:239-257](#)



```
def compute_score(self, q_reps, p_reps, q_mask, dense_weight=1.0, sparse_weight=0.3, colbert_weight=0.7):
    dense_score = self.compute_dense_score(q_reps, p_reps)
    sparse_score = self.compute_sparse_score(q_reps, p_reps)
    colbert_score = self.compute_colbert_score(q_reps, p_reps, q_mask=q_mask)
    return dense_score * dense_weight + sparse_score * sparse_weight + colbert_score * colbert_weight

def ensemble_score(self, q_reps, p_reps, dense_scores=None, sparse_scores=None, colbert_scores=None):
    if dense_scores is None or sparse_scores is None or colbert_scores is None:
        raise ValueError("dense_scores, sparse_scores, colbert_scores must be provided!")
    return dense_scores + 0.3 * sparse_scores + colbert_scores
```



```
def compute_score_single_device(self, sentence_pairs, ...):
    # ...
    dense_scores = self.model.compute_dense_score(q_dense_vecs, p_dense_vecs)
    sparse_scores = self.model.compute_sparse_score(q_sparse_vecs, p_sparse_vecs)
    colbert_scores = self.model.compute_colbert_score(
        q_colbert_vecs, p_colbert_vecs,
        q_mask=queries_inputs['attention_mask']
    )
    # ...  ...
```

7.

FlagEmbedding

		pooling
		/
		KL M3 KD
MRL		
CoIBERT &		M3

FlagEmbedding

07 research projects

FlagEmbedding

Mermaid Diagram

Embedding

Model	Size	Features	Performance	Notes
BGE-M3	1.1B	Text+Image+Audio	70.0	SOTA on MTEB
LM-Cocktail	1.1B	Text+Image	68.0	OpenAI GPT-4o
LLM-Embedder	1.1B	6 Multimodal	70.0	LLM-based
Activation-Beacon	1.1B	Text+Image	400K	OpenAI GPT-4o
BGE-VL	1.1B	MegaPairs	70.0	OpenAI GPT-4o
BGE-Code	1.1B	Text+Image	70.0	CoIR SOTA

Embedding

Mermaid Diagram

Table

- 1. BGE-M3
- 2. LM-Cocktail
- 3. LLM-Embedder LLM
- 4. Activation-Beacon
- 5. BGE-VL
- 6. BGE-Code
- 7. Embedding

BGE-M3

BGE-M3 FlagEmbedding "M3" **Multi-Functionality** **Multi-Linguality** **Multi-Granularity**

- [BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation](#)
-
- `/workspace/FlagEmbedding/inference/embedder/encoder_only/m3.py`
- `/workspace/FlagEmbedding/finetune/embedder/encoder_only/m3/`
- `/workspace/research/BGE_M3/`

1. Multi-Functionality

BGE-M3

1 **Dense Retrieval**

`/workspace/research/BGE_M3/modeling.py`

```
def dense_embedding(self, hidden_state, mask):
    if self.sentence_pooling_method == 'cls':
        return hidden_state[:, 0]
    elif self.sentence_pooling_method == 'mean':
        s = torch.sum(hidden_state * mask.unsqueeze(-1).float(), dim=1)
        d = mask.sum(axis=1, keepdim=True).float()
        return s / d
```

`/workspace/FlagEmbedding/inference/embedder/encoder_only/m3.py` - `cls` `mean` -

2 **Sparse Retrieval / Lexical Matching**

BM25 token

```

/workspace/research/BGE_M3/modeling.py

```

```
def sparse_embedding(self, hidden_state, input_ids, return_embedding: bool = True):
    token_weights = torch.relu(self.sparse_linear(hidden_state))
    if not return_embedding: return token_weights

    if self.training:
        sparse_embedding = torch.zeros(
            input_ids.size(0), input_ids.size(1), self.vocab_size,
            dtype=token_weights.dtype,
            device=token_weights.device
        )
        sparse_embedding = torch.scatter(sparse_embedding, dim=-1, index=input_ids.unsqueeze(-1),
                                         src=token_weights)
        sparse_embedding = torch.max(sparse_embedding, dim=1).values
    else:
        # 使用 scatter_reduce
        sparse_embedding = torch.zeros(
            input_ids.size(0), self.vocab_size,
            dtype=token_weights.dtype,
            device=token_weights.device
        )
        sparse_embedding = sparse_embedding.scatter_reduce(
            dim=-1, index=input_ids, src=token_weights.squeeze(-1), reduce="amax"
        )

    unused_tokens = [self.tokenizer.cls_token_id, self.tokenizer.eos_token_id,
                     self.tokenizer.pad_token_id, self.tokenizer.unk_token_id]
    sparse_embedding[:, unused_tokens] *= 0.
    return sparse_embedding
```

1000000 - 10 sparse_linear 1000000 token 10 - 10 ReLU 1000000 - 10 token [CLS] [EOS]
 PAD [UNK] 1000000 0 - 1000000 scatter reduce 1000000

Multi-Vector Retrieval / ColBERT

token

```

/workspace/research/BGE_M3/modeling.py

```

d_51bce0c78

```
/workspace/research/BGE_M3/modeling.py
```

Multi-Linguality

100 XLM-RoBERTa

- XXXXX XLM-RoBERTa-large
XXXXXX 250,000+
XXXXXXXXXXXXXXXXXXXX

Multi-Granularity

8192 tokens

```

/workspace/research/BGE_M3/modeling.py

```

 $16), (0, 3$ [illegible]

--	--	--	--	--	--

```
workspace /workspace/research/BGE_M3/modeling.py
```

```
def load_model(self, model_name, colbert_dim: int = -1):
    self.model = AutoModel.from_pretrained(model_name)
    self.tokenizer = AutoTokenizer.from_pretrained(model_name)

    # 初始化ColBERT线性层
    self.colbert_linear = torch.nn.Linear(in_features=self.model.config.hidden_size,
                                           out_features=self.model.config.hidden_size if colbert_dim == -1 else colbert_dim)
    self.sparse_linear = torch.nn.Linear(in_features=self.model.config.hidden_size, out_features=self.model.config.hidden_size)
```

```
workspace /workspace/research/BGE_M3/modeling.py
```

```
if self.unified_finetuning:
    # 计算稀疏得分和损失
    sparse_scores = self.sparse_score(q_sparse_vecs, p_sparse_vecs)
    sparse_loss = self.compute_loss(sparse_scores, targets)

    colbert_scores = self.colbert_score(q_colbert_vecs, p_colbert_vecs, q_mask=query['attention_mask'], p_mask=p_mask)
    colbert_loss = self.compute_loss(colbert_scores, targets)

    # 计算ensemble损失
    ensemble_loss = self.compute_loss(dense_scores + 0.3 * sparse_scores + colbert_scores, targets)

    # 计算总损失
    loss = (loss + ensemble_loss + 0.1 * sparse_loss + colbert_loss) / 4
```

Self-Knowledge Distillation

BGE-M3 

```
workspace /workspace/research/BGE_M3/modeling.py
```

```

if self.use_self_distill and self.step > self.self_distill_start_step and self.unified_finetuning:
    ensemble_scores = dense_scores + 0.3 * sparse_scores + colbert_scores
    teacher_targets = torch.softmax(ensemble_scores.detach(), dim=-1)

    ensemble_distill_dense_loss = - torch.mean(
        torch.sum(torch.log_softmax(dense_scores, dim=-1) * teacher_targets, dim=-1))
    ensemble_distill_sparse_loss = - torch.mean(
        torch.sum(torch.log_softmax(sparse_scores, dim=-1) * teacher_targets, dim=-1))
    ensemble_distill_colbert_loss = - torch.mean(
        torch.sum(torch.log_softmax(colbert_scores, dim=-1) * teacher_targets, dim=-1))

    loss += (ensemble_distill_dense_loss + 0.1 * ensemble_distill_sparse_loss + ensemble_distill_colbert_loss)
    loss = loss / 2

```

[] [] [] [] [] []

M3Embedder [] [] /workspace/FlagEmbedding/inference/embedder/encoder_only/m3.py [] []

[] [] [] 1. [] [] [] [] [] [] [] [] [] 2. [] [] [] [] [] [] [] [] [] 3. [] [] ColBERT [] [] [] [] 4. [] [] [] [] [] [] [] [] []

[] [] [] [] []

```

def encode_single_device(self, sentences, batch_size=256, max_length=8192,
                        return_dense=True, return_sparse=False, return_colbert_vecs=False):
    # ...
    # [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] padding [ ] [ ]
    length_sorted_idx = np.argsort([-len(x['input_ids']) for x in all_inputs])
    # ...
    # [ ] [ ] [ ] batch size [ ] [ ] OOM
    while flag is False:
        try:
            # ...
        except RuntimeError as e:
            batch_size = batch_size * 3 // 4

```

[] [] [] [] [] [] [] [] []

```
def compute_score_single_device(self, sentence_pairs, batch_size=256,
                                max_query_length=512, max_passage_length=512,
                                weights_for_different_modes=None):
    # ...
    # []
    if weights_for_different_modes is None:
        weights_for_different_modes = [1., 1., 1.]
        weight_sum = 3
    # ...
    all_scores['sparse+dense'] = (sparse_scores * w[1] + dense_scores * w[0]) / (w[1] + w[0])
    all_scores['colbert+sparse+dense'] = (colbert_scores * w[2] + sparse_scores * w[1] + dense_scores * w[0]) / (w[2] + w[1] + w[0])
```

LM-Cocktail

□ □ □ □

LM-Cocktail  1.  2. 


- [LM-Cocktail: Resilient Tuning of Language Models via Model Merging](#)
- `/workspace/research/LM_Cocktail/`

□ □ □ □

[illegible]

$$\text{Merged} = w_1 * M_1 + w_2 * M_2 + \dots + w_n * M_n$$

□ □ □ □

1.

```
../../../../workspace/research/LM_Cocktail/LM_Cocktail/cocktail.py
```



```
def mix_models_with_data(model_names_or_paths: List[str],
                          model_type: str,
                          example_data: List[Dict],
                          temperature: float=5.0,
                          batch_size:int=2,
                          max_input_length:int=2048,
                          neg_number: int=7,
                          output_path: str=None):

    """
    mix model based on given a few examples
    """

    # 1. 토큰화
    weights = compute_weights(model, tokenizer=tokenizer, param_list=param_list,
                              model_type=model_type, example_data=example_data,
                              temperature=temperature, neg_number=neg_number,
                              batch_size=batch_size, max_input_length=max_input_length)

    # 2. 모델 로드
    new_param = merge_param(param_list, weights=weights)
    model.load_state_dict(new_param)

    return model
```

3. □□□□□□□□□□

[illegible]

```

/workspace/research/LM_Cocktail/LM_Cocktail/cocktail.py

```

```
def mix_models_by_layers(model_names_or_paths: List[str],
                        model_type: str,
                        weights: List[float],
                        output_path: str=None):

    """
    mix models based on given weights, and load them layer by layer
    """

    # 创建临时目录
    param_dirs, temp_dir = get_model_param_dirs(model_names_or_paths, model_type=model_type)
    temp_file_path = merge_param_by_layer(param_dirs, weights=weights)

    # 使用 accelerate 加载模型
    with init_empty_weights():
        meta_model = AutoModelForCausalLM.from_pretrained(...) # 加载预训练模型

    device_map = {name: "cpu" for name, _ in meta_model.named_modules()}
    model = load_checkpoint_and_dispatch(meta_model, checkpoint=temp_file_path, device_map=device_map)

    # 清理临时文件
    os.remove(temp_file_path)
    shutil.rmtree(temp_dir)

    return model
```

--	--	--	--

1. 
2. 
3. 

□ □ □ □

- 
- 
- 

LLM-Embedder LLM

LLM-Embedder

- Retrieve Anything To Augment Large Language Models
- /workspace/research/llm_embedder/

LLM-Embedder 6

QA	
Conversational Search	
Long Conversation	
Long-Range Language Modeling	
In-Context Learning	
Tool Learning	

```
INSTRUCTIONS = {
  "qa": {
    "query": "Represent this query for retrieving relevant documents: ",
    "key": "Represent this document for retrieval: ",
  },
  "icl": {
    "query": "Convert this example into vector to look for useful examples: ",
    "key": "Convert this example into vector for retrieval: ",
  },
  # ...
}
```

--	--	--	--

- `llm-embedder` /workspace/research/llm_embedder/evaluation/
- `llm-embedder` BM25 `llm-embedder`

Activation-Beacon

□ □ □ □

Activation-Beacon

- Soaring from 4K to 400K: Extending LLM's Context with Activation Beacon
- /workspace/research/Long_LLM/activation_beacon/

□ □ □ □

LLM Beacon

1. 
2.  Beacon 
3.  Beacon 

📄📄📄

- 📄📄📄📄📄📄📄📄📄📄📄📄
- 📄📄📄📄📄📄📄📄
- 📄📄📄📄📄 LLM 📄📄Llama📄Mistral📄Qwen 📄📄
- 📄📄📄📄📄📄📄📄📄📄

📄📄📄📄

- Mistral
- Llama-3
- Qwen-2
- 📄📄📄📄📄📄

📄📄📄

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "namespace-Pt/beacon-qwen-2-7b-instruct"
tokenizer = AutoTokenizer.from_pretrained(model_id, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(model_id, trust_remote_code=True)

# 📄📄📄
model.memory.reset()

# 📄📄📄📄📄
messages = [{"role": "user", "content": very_long_text}]
inputs = tokenizer.apply_chat_template(messages, return_tensors="pt")
outputs = model.generate(**inputs)
```

BGE-VL📄📄📄📄📄📄

📄📄📄

BGE-VL 📄 BGE 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄

- 📄📄📄[MegaPairs: Massive Data Synthesis For Universal Multimodal Retrieval](#)

- `/workspace/research/BGE_VL/`

Table

Model	Embedder	Performance
BGE-VL-base	CLIP	
BGE-VL-large	CLIP	
BGE-VL-MLLM-S1	LLaVA-NeXT	on MegaPairs SOTA
BGE-VL-MLLM-S2	LLaVA-NeXT	S1 + MMEB

MegaPairs

MegaPairs 2600

- KNN + →
-
-

Steps

-
-
- + →
-

BGE-Code

Table

BGE-CodeCodeR

- [BAAI/bge-code-v1](#)
- `/workspace/research/BGE_Coder/`

特点

- 1. 在多个数据集上取得 SOTA
- 2. 支持多种语言
- 3. 支持 20+ 个数据集
- 4. 支持 4096 tokens

数据集

数据集	语言
MSR	中文
MSR	中文
MSR	中文
MSR	中文
MSR	中文
MSR	中文

应用

CoIRCode Retrieval BenchmarkCodeRAG BGE-Code

相关链接

1. BGE-Reranker & LLM-Reranker

-
- FlagEmbedding/inference/reranker/
- research/llm_reranker/
- 支持 LLM

2. C-MTEB & MLDR

- **C-MTEB**  MTEB 
- **MLDR**  13 
-  /workspace/research/C_MTEB/

3. LongLLM-QLoRA

□□ QLoRA □□□□ Llama-3 □□□□□□ 80K□

- `./workspace/research/Long_LLM/longllm_qlora/`
- `./workspace/research/Long_LLM/longllm_qlora/`

4. Visualized-BGE

token BGE

- 
- 

5. Matryoshka-Reranker

Matryoshka

6. LLARA

LLM

7. Reinforced-IR

11

[illegible]

1. BGE v1.5 M3 Code VL

- 2. LM-Cocktail Activation-Beacon
- 3. LLM-Embedder LLM Reranker
- 4. MegaPairs MLDR C-MTEB

08 model survey

FlagEmbedding

FlagEmbedding Embedder Reranker

-
-
- Embedder
- BGE
- Qwen3-Embedding
- E5
- GTE
- SFR
- Linq
- BCE
- Reranker
- BGE
- Jina
- GTE
- BCE
-

-
-

FlagEmbedding

FlagEmbedding BAAI RAG

- API - - -

1. FlagEmbedding 2. 3. 4.

Mermaid Diagram

Embedder

BGE

BGE BAAI General Embedding FlagEmbedding

BGE v1.0

bge-large-en	English	Large	
bge-base-en	English	Base	
bge-small-en	English	Small	
bge-large-zh	Chinese	Large	
bge-base-zh	Chinese	Base	
bge-small-zh	Chinese	Small	

Encoder-only - CLS pooling - Represent this sentence for searching relevant passages:

```
from FlagEmbedding import FlagAutoModel

model = FlagAutoModel.from_finetuned('BAAI/bge-large-en-v1.5')
```

BGE v1.5

v1.5 v1.0

bge-large-en-v1.5	English	Large	
bge-base-en-v1.5	English	Base	
bge-small-en-v1.5	English	Small	
bge-large-zh-v1.5	Chinese	Large	
bge-base-zh-v1.5	Chinese	Base	
bge-small-zh-v1.5	Chinese	Small	

1. 2. 3. MTEB C-MTEB

BGE-M3

bge-m3

- **Multi-function** - **Multilingual** 100+ - **Multi-granularity** 8192

- Encoder-only - -

1. **Dense Retrieval** 2. **Sparse Retrieval** 3. **ColBERT** token

- - - -

```
from FlagEmbedding import BGEM3FlagModel

model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)

# 
embeddings = model.encode([''], return_dense=True, return_sparse=True, return_colbert_vecs=True)
```

[FlagEmbedding/inference/embedder/encoder_only/m3.py](#)

BGE

1. bge-code-v1

- Decoder-only - Last token pooling - `<instruct>{} \n<query>{}`

- - -

2. bge-en-icl

- In-Context Learning - Decoder-only - `<instruct>{} \n<query>{}`

- - -

3. bge-multilingual-gemma2

Model - Gemma-2-9b - Decoder-only - MIRACL, MTEB-fr, MTEB-pl

4. bge-reasoner-embed-qwen3-8b-0923

Model - Qwen3-8B - Instruct: {} \n Query: {}

FlagEmbedding/inference/embedder/model_mapping.py

Qwen3-Embedding

Qwen3-Embedding Qwen3

			Pooling
Qwen3-Embedding-0.6B	0.6B	Decoder-only	Last token
Qwen3-Embedding-4B	4B	Decoder-only	Last token
Qwen3-Embedding-8B	8B	Decoder-only	Last token

Model - Qwen3 - Decoder-only - Instruct: {} \n Query:{} -

Model - - -

FlagEmbedding/inference/embedder/model_mapping.py

E5

E5Embedding v5 Microsoft

00 E5 00

0000	00	Pooling
e5-large	Large	Mean
e5-base	Base	Mean
e5-small	Small	Mean

E5 v2 00

0000	00	00
e5-large-v2	Large	0000
e5-base-v2	Base	0000
e5-small-v2	Small	0000

E5-Mistral-7B

000 - 00 Mistral-7B - Decoder-only 00 - Last token pooling - 000000 Instruct: {}
\nQuery: {}

Multilingual E5 00

0000	00
multilingual-e5-large	000000
multilingual-e5-base	0000000
multilingual-e5-small	000000
multilingual-e5-large-instruct	0000000000

000000 - 0000000000 - 000000 - 000000

000000 [FlagEmbedding/inference/embedder/model_mapping.py](#)

GTE ☐ ☐ ☐ ☐

GTE□General Text Embedding□□□□□□□□□□

GTE ☐ ☐ ☐

数据集	语言	Pooling
gte-large	English	Mean
gte-base	English	Mean
gte-small	English	Mean
gte-large-zh	Chinese	CLS
gte-base-zh	Chinese	CLS
gte-small-zh	Chinese	CLS

GTE v1.5

Embedding	Model
gte-large-en-v1.5	Model v1.5
gte-base-en-v1.5	Model v1.5

GTE-Qwen

Embedding	Model
gte-Qwen2-7B-instruct	Qwen2-7B Embedding
gte-Qwen2-1.5B-instruct	Qwen2-1.5B Embedding
gte-Qwen1.5-7B-instruct	Qwen1.5-7B Embedding

GTE

gte-multilingual-base	

- - - GTE-Qwen

- - -

[FlagEmbedding/inference/embedder/model_mapping.py](#)

SFR

SFRSalesforce RepresentationsSalesforce

SFR-Embedding-Mistral	Mistral
SFR-Embedding-2_R	SFR

- Decoder-only - Last token pooling - Instruct: {} \nQuery: {}

- - Salesforce -

[FlagEmbedding/inference/embedder/model_mapping.py](#)

Linq

Linq-Embed-Mistral	Mistral Linq

- Decoder-only - Last token pooling - Instruct: {} \nQuery: {}

- - Linq

[FlagEmbedding/inference/embedder/model_mapping.py](#)

BCE

Model Name	Architecture
bce-embedding-base_v1	BCE

Encoder-only - CLS pooling

- BCE

[FlagEmbedding/inference/embedder/model_mapping.py](#)

Reranker

Reranker

BGE

BGE FlagEmbedding

Model Name	Architecture	Details
bge-reranker-base	Encoder-only	
bge-reranker-large	Encoder-only	
bge-reranker-v2-m3	Encoder-only	
bge-reranker-v2-gemma	Decoder-only	Gemma
bge-reranker-v2-minicpm-layerwise	Decoder-only	
bge-reranker-v2.5-gemma2-lightweight	Decoder-only	

1. bge-reranker-base & large

Encoder-only - Cross-encoder -

- -

2. bge-reranker-v2-m3

🔗 - 📄📄📄📄 - 📄📄📄📄📄 - 📄📄 - 📄📄

3. bge-reranker-v2-gemma

🔗 - 📄 Gemma 📄 - 📄📄 - 📄📄📄📄📄

4. bge-reranker-v2-minicpm-layerwise

🔗 - 📄Layer-wise📄 - 📄📄📄📄📄 - 📄📄 - 📄📄📄

5. bge-reranker-v2.5-gemma2-lightweight

🔗 - 📄 Gemma-2 - 📄📄 - 📄📄📄 - 📄📄 - 📄📄📄📄📄

📄📄📄

```
from FlagEmbedding import FlagAutoReranker

reranker = FlagAutoReranker.from_finetuned('BAAI/bge-reranker-large')

scores = reranker.compute_score([
    ('📄', '📄1'),
    ('📄', '📄2')
])
```

📄📄📄📄 [FlagEmbedding/inference/reranker/model_mapping.py](#)

Jina 🔗🔗🔗🔗

📄📄	📄
jinaai/jina-reranker-v2-base-multilingual	📄📄📄📄📄
jinaai/jina-reranker-v1-turbo-en	📄 Turbo 📄

📄📄 - Encoder-only 📄 - Jina AI 📄 - 📄📄

📄📄📄 - 📄📄 - Jina 📄📄📄 - 📄📄📄📄

📄📄📄 [FlagEmbedding/inference/reranker/model_mapping.py](#)

GTE 模型

模型	名称
Alibaba-NLP/gte-multilingual-reranker-base	GTE 模型

模型 - Encoder-only 模型 - 模型 - 模型

模型 - 模型 - 模型 - 模型

模型 [FlagEmbedding/inference/reranker/model_mapping.py](#)

BCE 模型

模型	名称
maidalun1020/bce-reranker-base_v1	BCE 模型

模型 - Encoder-only 模型 - BCE 模型

模型 - 模型 - BCE 模型

模型 [FlagEmbedding/inference/reranker/model_mapping.py](#)

模型

模型

Mermaid Diagram

Embedding

Model	Embedding	Features
Embedding	bge-v1.5, E5, GTE	Embedding
Embedding	bge-m3	Embedding+Embedding+Embedding
Embedding	bge-en-icl	Embedding In-Context Learning
Embedding	bge-code-v1	Embedding
Embedding	bge-m3, bge-multilingual-gemma2	Embedding
Embedding	bge-m3 (8192)	Embedding

Embedding

Embedding

Model	Embedding	Embedding	Embedding	Embedding	Embedding	Embedding
BGE v1.5 Small	Small	Embedding	Embedding	Embedding	Embedding	Embedding
BGE v1.5 Base	Base	Embedding	Embedding	Embedding	Embedding	Embedding
BGE v1.5 Large	Large	Embedding	Embedding	Embedding	Embedding	Embedding
BGE-M3	Large	Embedding	Embedding	Embedding	Embedding	Embedding
Qwen3-Embedding-0.6B	0.6B	Embedding	Embedding	Embedding	Embedding	Embedding
Qwen3-Embedding-8B	8B	Embedding	Embedding	Embedding	Embedding	Embedding
E5 Small	Small	Embedding	Embedding	Embedding	Embedding	Embedding
E5 Large	Large	Embedding	Embedding	Embedding	Embedding	Embedding
GTE	Mixed	Embedding	Embedding	Embedding	Embedding	Embedding
SFR	7B	Embedding	Embedding	Embedding	Embedding	Embedding

	📄
base	Encoder-only
large	Encoder-only
v2-m3	Encoder-only
v2-gemma	Decoder-only
v2-minicpm-	Decoder-only
v2.5-gemma2-	Decoder-only

10 *Journal of Maritime Law and Commerce*

□□	□□
GTE v1/ 1.5, GTE-M3, GTE, GTE	□□ <[BOS_never_used_51bce0 token □□□□□□
□, GTE □	□□□□ token □□□□□
decoder- only □□	□□□□□□ token□□□□□□□□

□□□□□ [FlagEmbedding/inference/embedder/model_mapping.py](#)

□□□□□□□

□□□□□□□□□

□□ **1**□□□□□□□□

□□□□□□ - □□□□□ bge-large-en-v1.5 □ e5-large-v2 - □□□□□ bge-base-en-v1.5 - □□□□□ bge-small-en-v1.5

□□□□□□□ bge-reranker-large

□□ **2**□□□□□□□□

□□□□□□ - □□□□□ bge-large-zh-v1.5 - □□□□□ bge-base-zh-v1.5 - □□□□□ bge-small-zh-v1.5 □ Qwen3-Embedding-0.6B

□□□□□□□ bge-reranker-large

□□ **3**□□□□□□□

□□□□□□ - □□□□ bge-m3 □□□□□ + □□□□ + □□□□□ - □□□□ bge-multilingual-gemma2 □ multilingual-e5-large

□□□□□□□ bge-reranker-v2-m3

□□ **4**□□□□□□□

□□□□□□ - bge-m3 □□□□□ 8192

□□□□□□□ bge-reranker-v2-gemma □ bge-reranker-v2.5-gemma2-lightweight

□□ **5**□□□□□□

□□□□□□ - bge-code-v1

□□ **6**□□□□/□□□□□□

□□□□□□ - GTE □□□□ gte-large-zh □ gte-multilingual-base - □□□□ bge-m3

安装 gte-multilingual-reranker-base

7 [1]

安装 - bge-en-icl [1]

[1]

安装 CPU/[1]

安装 - [1] bge-small-en-v1.5 / bge-small-zh-v1.5 / Qwen3-Embedding-0.6B - [1]
bge-reranker-v2-m3 [1] bge-reranker-v2.5-gemma2-lightweight - [1] use_fp16=True

安装 GPU [1]

安装 - [1] bge-base-en-v1.5 / bge-base-zh-v1.5 - [1] bge-reranker-base [1] bge-reranker-v2-minicpm-layerwise - [1] use_fp16=True [1] batch size

安装 GPU [1]

安装 - [1] bge-large-en-v1.5 / bge-large-zh-v1.5 / bge-m3 - [1] bge-reranker-large [1] bge-reranker-v2-gemma - [1] batch size

[1]

1. [1]

Mermaid Diagram

[1] - [1] - [1] - [1]

2. [1]

[1]

```
# BGE v1.5 
query_instruction = "Represent this sentence for searching relevant passages: "

# BGE v1.5 
query_instruction = "你是一名乐于助人的AI助手，请回答以下问题。"

# Decoder-only 
query_instruction = "Instruct: {} \n Query: {}"
```

3.



```
# 
embeddings = model.encode(large_corpus, batch_size=256)

# 
for doc in large_corpus:
    embedding = model.encode(doc) # 
```

4.



```
model = FlagAutoModel.from_finetuned(
    'BAAI/bge-large-en-v1.5',
    devices=[0, 1, 2, 3] #  GPU
)
```

5.

 bge-m3

```
# 1. Dense
dense_scores = dense_embeddings @ query_dense_embedding.T

# 2. Sparse
sparse_scores = compute_sparse_similarity(sparse_embeddings, query_sparse_embedding)

# 3. Colbert
colbert_scores = compute_colbert_similarity(colbert_vecs, query_colbert_vecs)

# 4. Fusion
final_scores = 0.6 * dense_scores + 0.2 * sparse_scores + 0.2 * colbert_scores
```



FlagEmbedding

FlagEmbedding 

1. 
2. 7  BGE  Qwen3  E5  GTE  SFR  Linq  BCE 
3. 4  BGE  Jina  GTE  BCE 
4.  Encoder-only  Decoder-only 
5. 
6. 
7. 
8. 
9. 
10. 
11. 
12. 



1. 
2. 
3. 
4. 



 FlagEmbedding 

1. 
2. 
3. 
4. 
5. 
6. 
7. 
8. 
9. 
10. 
11. 
12. 
13. 
14. 
15. 
16. 



FlagEmbedding 

1.  BGE 
2. 

3. 项目地址: [https://github.com/FlagOpen/FlagEmbedding](#)

4. 项目文档: [https://flagembedding.com](#)

项目

项目地址: [https://github.com/FlagOpen/FlagEmbedding](#)

项目地址: [https://github.com/FlagOpen/FlagEmbedding](#) **40+** 项目

项目地址: [https://github.com/FlagOpen/FlagEmbedding/inference/embedder/model_mapping.py](#)

项目地址: [https://github.com/FlagOpen/FlagEmbedding](#) **9** 项目

项目地址: [https://github.com/FlagOpen/FlagEmbedding/inference/reranker/model_mapping.py](#)

项目地址:

- [GitHub](#)
- [HuggingFace](#) 项目
- [项目](#)
- [Notebook](#)

项目地址:

1. C-Pack: Packaged Resources To Advance General Chinese Embedding (2023)
2. LM-Cocktail: Resilient Tuning of Language Models via Model Merging (2023)
3. Retrieve Anything To Augment Large Language Models (2023)
4. BGE-M3: Multi-lingual, Multi-functional, Multi-granularity Embedding (2024)

项目地址: [https://github.com/FlagOpen/FlagEmbedding](#) v1.1 项目地址: [https://github.com/FlagOpen/FlagEmbedding](#) 2024
